

DrugResponsePredictionGroup2K61FINAL

May 16, 2026

1 Machine Learning-Based Prediction of Cancer Drug Response

Big Data in Bioinformatics (BIF 324)

Section: K61 Lamees Yasser 445007462

Raghad Althobaiti 445007474

Renad Alsaud 445007493

Jood Almaymouni 445007479

Anar Aldhwacla 445007488

Reemas Alanzi 445007458

Shahad Alduwayrij 445007470

Project Overview: This project aims to develop machine learning models for predicting cancer drug response using gene expression data from the Genomics of Drug Sensitivity in Cancer (GDSC) database. The target of the analysis is drug sensitivity, represented by IC50 values, which indicate how responsive cancer cell lines are to specific drugs. The workflow includes data preprocessing, feature engineering, exploratory data analysis, predictive model development, model evaluation, feature importance analysis, and validation of drug response predictions using appropriate statistical and machine learning approaches. The project may help identify genomic patterns and important genes associated with drug sensitivity, supporting a better understanding of treatment response and personalized medicine approaches.

Import Libraries

```
[7]: !pip install xgboost
      # Install xgboost library
```

```
Requirement already satisfied: xgboost in c:\users\ja\anaconda3\lib\site-
packages (3.2.0)
```

```
Requirement already satisfied: numpy in c:\users\ja\anaconda3\lib\site-packages
(from xgboost) (1.26.4)
```

```
Requirement already satisfied: scipy in c:\users\ja\anaconda3\lib\site-packages
(from xgboost) (1.13.1)
```

```
[1]: # Data handling
      import pandas as pd
```

```

import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Saving and loading trained machine learning models
import joblib

# Preprocessing and splitting
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

# Regression models
from sklearn.linear_model import Ridge, LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV

# Regression evaluation metrics
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

```

```

[3]: sns.set_theme()
     # Plot style

```

Load, Explore and Merge the Datasets Jood Almaymouni

```

[15]: Gene_Expression = pd.read_csv("rnaseq_merged_rsem_tpm_20260323.csv", low_memory_
    ↪ False)
     # Load the gene expression dataset

```

```

[16]: Gene_Expression.head()
     # Display the first five rows of the dataset

```

```

[16]:
   model_id  Unnamed: 1 Unnamed: 2 SIDM00001 SIDM00002 SIDM00003 \
0  model_name          NaN         NaN    MEC-1    NBSusSR         M14
1  data_source          NaN         NaN    Broad    Sanger         Sanger
2  gene_symbol  ensembl_gene_id  gene_id      NaN      NaN         NaN
3      A1BG  ENSG00000121410  SIDG00001    3.853    4.5059    3.9736
4    A1BG-AS1  ENSG00000268895  SIDG00002    1.8156    2.3868    3.1408

   SIDM00005 SIDM00006 SIDM00007 SIDM00008 ...  SIDM02262 \
0  MDA-MB-134-VI    MCC26    MCC13    MCAS ...  WTSI-COLO_427
1      Broad    Sanger    Sanger    Broad ...      Sanger
2      NaN      NaN      NaN      NaN ...      NaN
3    5.7038    3.9069    6.1198    0.0566 ...    0.1635
4    3.3334    2.5801    3.5336    0.0976 ...      0

```

	SIDM02263	SIDM02264	SIDM02265	SIDM02266 \
0	WTSI-COLO_425	WTSI-COLO_061	WTSI-COLIVM_100	WTSI-COLIVM_093
1	Sanger	Sanger	Sanger	Sanger
2	NaN	NaN	NaN	NaN
3	0.848	0.2869	0.189	0.5361
4	0	0.189	0.0976	0.2141

	SIDM02267	SIDM02268	SIDM02269	SIDM02270 \
0	WTSI-COLIVM_092	WTSI-COLO_452	WTSI-PANC_131	WTSI-COLIVM_068
1	Sanger	Sanger	Sanger	Sanger
2	NaN	NaN	NaN	NaN
3	3.5583	2.9635	0	0.0976
4	0.4957	0.3334	0.8953	0.2869

	SIDM02271
0	WTSI-GAST_200
1	Sanger
2	NaN
3	0
4	0

[5 rows x 1901 columns]

```
[17]: Gene_Expression.tail()
# Display the last five rows of the dataset
```

```
[17]:
```

	model_id	Unnamed: 1	Unnamed: 2	SIDM00001	SIDM00002	SIDM00003 \
41143	ZYG11B	ENSG00000162378	SIDG42471	1.926	3.7388	3.7148
41144	ZYX	ENSG00000159840	SIDG42472	4.1562	5.2687	6.959
41145	ZYXP1	ENSG00000274572	SIDG42473	NaN	0	0
41146	ZZEF1	ENSG00000074755	SIDG42474	3.0959	4.3519	4.3561
41147	ZZZ3	ENSG00000036549	SIDG42475	3.6792	5.4182	5.5622

	SIDM00005	SIDM00006	SIDM00007	SIDM00008 ...	SIDM02262	SIDM02263 \
41143	2.4463	3.6334	2.9241	2.121 ...	1.251	1.433
41144	4.0009	8.5417	7.5178	6.3811 ...	3.5398	3.9011
41145	NaN	0	0	NaN ...	0	0
41146	3.3813	3.5211	3.5874	2.4725 ...	0.5059	1.2265
41147	4.2434	5.1839	4.9435	4.0721 ...	2.9819	2.5361

	SIDM02264	SIDM02265	SIDM02266	SIDM02267	SIDM02268	SIDM02269	SIDM02270 \
41143	2.4303	1.6276	2.6485	1.0286	2.3448	1.111	3.2854
41144	6.4499	6.5396	7.2235	4.1587	4.8802	3.4101	6.7605
41145	0	0	0	0	0	0	0
41146	1.8156	1.8237	2.6508	1.4542	0.9928	0.9561	2.8237
41147	3.696	3.1408	3.1408	2.3045	3.5801	2.4725	4.4269

```

        SIDM02271
41143      2.0426
41144      5.8689
41145         0
41146      2.7949
41147      3.6496

```

[5 rows x 1901 columns]

```
[18]: Gene_Expression.shape
      # Check dataset dimensions
```

```
[18]: (41148, 1901)
```

```
[19]: Gene_Expression.info()
      # Display dataset information
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41148 entries, 0 to 41147
Columns: 1901 entries, model_id to SIDM02271
dtypes: object(1901)
memory usage: 596.8+ MB

```

```
[20]: Expression_Metadata = Gene_Expression.iloc[:3]
      # Seperate metadata rows from the gene expression matrix

      Expression_Matrix = Gene_Expression.iloc[3:].reset_index(drop = True)
      # Keep only rows containing actual gene expression values
```

```
[21]: Expression_Matrix = Expression_Matrix.drop(columns=["Unnamed: 1", "Unnamed: 2"])
      # Keep only useful columns
```

```
[22]: Expression_Matrix = Expression_Matrix.set_index("model_id")
      # Set gene names as index
```

```
[23]: Expression_Matrix.head()
      # Preview the cleaned expression matrix
```

```
[23]:
```

	SIDM00001	SIDM00002	SIDM00003	SIDM00005	SIDM00006	SIDM00007	\
model_id							
A1BG	3.853	4.5059	3.9736	5.7038	3.9069	6.1198	
A1BG-AS1	1.8156	2.3868	3.1408	3.3334	2.5801	3.5336	
A1CF	0	0.0286	0.0144	0.0566	0.0426	0.0144	
A2M	0.2388	0.5753	4.1326	0.5261	5.6562	0.0976	
A2M-AS1	0	0.3448	0.4114	1.4489	0.5656	0.6599	
	SIDM00008	SIDM00009	SIDM00011	SIDM00014	...	SIDM02262	SIDM02263 \

model_id					...		
A1BG	0.0566	4.5638	5.1961	0.0976	...	0.1635	0.848
A1BG-AS1	0.0976	3.1875	3.557	0.263	...	0	0
A1CF	0.0426	0.0144	0	0.7137	...	1.281	1.1953
A2M	0.0976	0.0841	7.6094	0.111	...	0.0566	0
A2M-AS1	0.0286	0.0286	1.5656	0.189	...	0.1506	0

	SIDM02264	SIDM02265	SIDM02266	SIDM02267	SIDM02268	SIDM02269	\
model_id							
A1BG	0.2869	0.189	0.5361	3.5583	2.9635		0
A1BG-AS1	0.189	0.0976	0.2141	0.4957	0.3334	0.8953	
A1CF	0.263	2.2265	1.5753	1.9336	2.0144		0
A2M	0.111	0.0566	0.0841	0	0.263	0.3448	
A2M-AS1	0.1635	0.0566	0.0286	0.0566	0.275	0.6599	

	SIDM02270	SIDM02271
model_id		
A1BG	0.0976	0
A1BG-AS1	0.2869	0
A1CF	2.2079	0.0144
A2M	0.4854	0.2141
A2M-AS1	0.3448	0.3896

[5 rows x 1898 columns]

```
[24]: Expression_Matrix.shape
      # Check dimensions after removing metadata rows
```

```
[24]: (41145, 1898)
```

```
[25]: Expression_Matrix.columns
      # Display column names
```

```
[25]: Index(['SIDM00001', 'SIDM00002', 'SIDM00003', 'SIDM00005', 'SIDM00006',
            'SIDM00007', 'SIDM00008', 'SIDM00009', 'SIDM00011', 'SIDM00014',
            ...,
            'SIDM02262', 'SIDM02263', 'SIDM02264', 'SIDM02265', 'SIDM02266',
            'SIDM02267', 'SIDM02268', 'SIDM02269', 'SIDM02270', 'SIDM02271'],
            dtype='object', length=1898)
```

```
[26]: Drug_Response = pd.read_excel("GDSC2_fitted_dose_response_27Oct23.xlsx")
      # Load the drug response dataset
```

```
[27]: Drug_Response.head()
      # Display the first five rows of the dataset
```

```
[27]: DATASET NLME_RESULT_ID NLME_CURVE_ID CELL_LINE_NAME SANGER_MODEL_ID \
0 GDSC2 343 15946310 PFSK-1 SIDM01132
1 GDSC2 343 15946548 A673 SIDM00848
2 GDSC2 343 15946830 ES5 SIDM00263
3 GDSC2 343 15947087 ES7 SIDM00269
4 GDSC2 343 15947369 EW-11 SIDM00203

CANCER_TYPE DRUG_ID DRUG_NAME PUTATIVE_TARGET \
0 Other Solid Cancers 1003 Camptothecin TOP1
1 Ewing's Sarcoma 1003 Camptothecin TOP1
2 Ewing's Sarcoma 1003 Camptothecin TOP1
3 Ewing's Sarcoma 1003 Camptothecin TOP1
4 Ewing's Sarcoma 1003 Camptothecin TOP1

PATHWAY_NAME MIN_CONC MAX_CONC LN_IC50 AUC RMSE Z_SCORE
0 DNA replication 0.0001 0.1 -1.463887 0.930220 0.089052 0.433123
1 DNA replication 0.0001 0.1 -4.869455 0.614970 0.111351 -1.421100
2 DNA replication 0.0001 0.1 -3.360586 0.791072 0.142855 -0.599569
3 DNA replication 0.0001 0.1 -5.044940 0.592660 0.135539 -1.516647
4 DNA replication 0.0001 0.1 -3.741991 0.734047 0.128059 -0.807232
```

```
[28]: Drug_Response.tail()
# Display the last five rows of the dataset
```

```
[28]: DATASET NLME_RESULT_ID NLME_CURVE_ID CELL_LINE_NAME SANGER_MODEL_ID \
242031 GDSC2 343 16188242 SNU-175 SIDM00216
242032 GDSC2 343 16188695 SNU-407 SIDM00214
242033 GDSC2 343 16188953 SNU-61 SIDM00194
242034 GDSC2 343 16189493 SNU-C5 SIDM00498
242035 GDSC2 343 16189775 DiFi SIDM00049

CANCER_TYPE DRUG_ID DRUG_NAME PUTATIVE_TARGET \
242031 Colorectal Carcinoma 2499 N-acetyl cysteine Metabolism
242032 Colorectal Carcinoma 2499 N-acetyl cysteine Metabolism
242033 Colorectal Carcinoma 2499 N-acetyl cysteine Metabolism
242034 Colorectal Carcinoma 2499 N-acetyl cysteine Metabolism
242035 Colorectal Carcinoma 2499 N-acetyl cysteine Metabolism

PATHWAY_NAME MIN_CONC MAX_CONC LN_IC50 AUC RMSE \
242031 Metabolism 2.001054 2000.0 10.127082 0.976746 0.074498
242032 Metabolism 2.001054 2000.0 8.576377 0.913378 0.057821
242033 Metabolism 2.001054 2000.0 10.519636 0.975001 0.058090
242034 Metabolism 2.001054 2000.0 10.694579 0.969969 0.101013
242035 Metabolism 2.001054 2000.0 10.034825 0.966988 0.089057

Z_SCORE
242031 0.156872
```

```
242032 -1.626959
242033  0.608442
242034  0.809684
242035  0.050746
```

```
[29]: Drug_Response.shape
      # Check dataset dimensions
```

```
[29]: (242036, 16)
```

```
[30]: Drug_Response.columns
      # Display column names
```

```
[30]: Index(['DATASET', 'NLME_RESULT_ID', 'NLME_CURVE_ID', 'CELL_LINE_NAME',
            'SANGER_MODEL_ID', 'CANCER_TYPE', 'DRUG_ID', 'DRUG_NAME',
            'PUTATIVE_TARGET', 'PATHWAY_NAME', 'MIN_CONC', 'MAX_CONC', 'LN_IC50',
            'AUC', 'RMSE', 'Z_SCORE'],
            dtype='object')
```

```
[31]: Expression_Transposed = Expression_Matrix.T
      # Transpose the expression matrix so cell lines become rows and genes become
      ↪ columns
```

```
[32]: Expression_Transposed = Expression_Transposed.reset_index()
      # Convert the cell line IDs from index into a normal column
```

```
[33]: Expression_Transposed = Expression_Transposed.rename(columns={"index":
      ↪ "SANGER_MODEL_ID"})
      # Rename the cell line ID column to match the drug response dataset
```

```
[34]: Expression_Transposed.head()
      # Preview transposed expression matrix
```

```
[34]: model_id SANGER_MODEL_ID    A1BG A1BG-AS1    A1CF    A2M A2M-AS1    A2ML1 \
0          SIDM00001    3.853    1.8156    0  0.2388    0    0
1          SIDM00002    4.5059    2.3868  0.0286  0.5753  0.3448  0.4751
2          SIDM00003    3.9736    3.1408  0.0144  4.1326  0.4114    0
3          SIDM00005    5.7038    3.3334  0.0566  0.5261  1.4489  0.4114
4          SIDM00006    3.9069    2.5801  0.0426  5.6562  0.5656  0.1506
```

```
model_id A2ML1-AS1 A2ML1-AS2    A2MP1 ...    ZXDA    ZXDB    ZXDC  ZYG11A \
0          0          0          0 ...  0.7655  1.9561  2.8992  0.8953
1          0          0  0.0144 ...  1.7137  2.6803  5.6156  0.848
2          0          0          0 ...  1.0909  2.101  4.0488  2.6369
3          0          0  0.0426 ...  1.7093  2.257  4.3111  1.0909
4          0          0  0.0286 ...  1.4005  2.3533  3.9551  2.98
```

```
model_id ZYG11AP1  ZYG11B    ZYX ZYXP1    ZZEF1    ZZZ3
```

```

0          0  1.926  4.1562  NaN  3.0959  3.6792
1          0  3.7388  5.2687    0  4.3519  5.4182
2      0.0976  3.7148  6.959    0  4.3561  5.5622
3          0  2.4463  4.0009  NaN  3.3813  4.2434
4          0  3.6334  8.5417    0  3.5211  5.1839

```

[5 rows x 41146 columns]

```

[35]: Expression_Transposed.columns[:10]
      # Check Columns

```

```

[35]: Index(['SANGER_MODEL_ID', 'A1BG', 'A1BG-AS1', 'A1CF', 'A2M', 'A2M-AS1',
            'A2ML1', 'A2ML1-AS1', 'A2ML1-AS2', 'A2MP1'],
            dtype='object', name='model_id')

```

```

[36]: Expression_Transposed.columns.name = None
      # Remove the column-axis name (cosmetic fix)

```

```

[37]: Expression_Transposed.head()
      # Check head after editing

```

```

[37]: SANGER_MODEL_ID  A1BG A1BG-AS1  A1CF  A2M A2M-AS1  A2ML1 A2ML1-AS1  \
0      SIDM00001    3.853   1.8156    0  0.2388    0    0    0
1      SIDM00002    4.5059   2.3868  0.0286  0.5753  0.3448  0.4751    0
2      SIDM00003    3.9736   3.1408  0.0144  4.1326  0.4114    0    0
3      SIDM00005    5.7038   3.3334  0.0566  0.5261  1.4489  0.4114    0
4      SIDM00006    3.9069   2.5801  0.0426  5.6562  0.5656  0.1506    0

```

```

      A2ML1-AS2  A2MP1  ...  ZXDA  ZXDB  ZXDC  ZYG11A ZYG11AP1  ZYG11B  \
0          0    0  ...  0.7655  1.9561  2.8992  0.8953    0  1.926
1          0  0.0144  ...  1.7137  2.6803  5.6156  0.848    0  3.7388
2          0    0  ...  1.0909  2.101  4.0488  2.6369  0.0976  3.7148
3          0  0.0426  ...  1.7093  2.257  4.3111  1.0909    0  2.4463
4          0  0.0286  ...  1.4005  2.3533  3.9551  2.98    0  3.6334

```

```

      ZYX ZYXP1  ZZEF1  ZZZ3
0  4.1562  NaN  3.0959  3.6792
1  5.2687    0  4.3519  5.4182
2   6.959    0  4.3561  5.5622
3  4.0009  NaN  3.3813  4.2434
4  8.5417    0  3.5211  5.1839

```

[5 rows x 41146 columns]

```

[38]: Expression_Transposed.shape
      # Check dimensions

```



```
[38]: (1898, 41146)
```

```
[39]: Drug_Response["PATHWAY_NAME"].value_counts()
      # Display the most common drug pathways
```

```
[39]: PATHWAY_NAME
Unclassified                24979
PI3K/MTOR signaling         22724
Other                       21402
DNA replication             17650
Other, kinases              17277
ERK MAPK signaling         13350
Genome integrity            12221
Cell cycle                  11620
Apoptosis regulation        10828
Chromatin histone methylation 10612
RTK signaling               10573
Chromatin other             8576
Chromatin histone acetylation 8162
WNT signaling               7631
Protein stability and degradation 7087
EGFR signaling              6734
Mitosis                    6453
IGF1R signaling             5512
Metabolism                  4802
Hormone-related             3820
p53 pathway                 3776
Cytoskeleton                3387
JNK and p38 signaling        1905
ABL signaling                955
Name: count, dtype: int64
```

```
[40]: Drug_Filtered = Drug_Response[Drug_Response["PATHWAY_NAME"] == "RTK signaling"]
      # Filter drugs belonging to the selected pathway
```

```
[41]: Drug_Filtered = Drug_Filtered[["SANGER_MODEL_ID", "DRUG_ID", "DRUG_NAME", "LN_IC50", "PATHWAY_NAME"]]
      # Select important columns for drug response prediction
```

```
[42]: Drug_Filtered.shape
      # Check filtered dataset dimensions
```

```
[42]: (10573, 5)
```

```
[43]: Metadata_Columns = Expression_Transposed[["SANGER_MODEL_ID"]]

Gene_Features = Expression_Transposed.drop(columns=["SANGER_MODEL_ID"])
```

```
# Separate identifier columns from gene expression features
```

```
[44]: Gene_Features = Gene_Features.apply(pd.to_numeric, errors="coerce")
# Convert gene expression values to numeric format
```

```
[45]: Gene_Variance = Gene_Features.var()
# Calculate variance
```

```
[46]: Top_Genes = Gene_Variance.sort_values(ascending=False).head(10000).index
# Select the 10,000 most variable genes
```

```
[47]: Reduced_Expression = Gene_Features[Top_Genes]
# Create reduced gene expression dataset
```

```
[48]: Reduced_Expression = pd.concat([Metadata_Columns, Reduced_Expression], axis=1)
# Add cell line identifiers back to the dataset
```

```
[49]: Reduced_Expression.shape
# Check reduced dataset dimensions
```

```
[49]: (1898, 10003)
```

```
[50]: Merged_Data = Drug_Filtered.merge(Reduced_Expression, on="SANGER_MODEL_ID",
↳how="inner")
# Merge expression data with drug response
```

```
[51]: Merged_Data
# Check dataset
```

```
[51]:
```

	SANGER_MODEL_ID	DRUG_ID	DRUG_NAME	LN_IC50	PATHWAY_NAME	VIM	\
0	SIDM00848	1021	Axitinib	1.180880	RTK signaling	10.5437	
1	SIDM00263	1021	Axitinib	0.767377	RTK signaling	7.3118	
2	SIDM00269	1021	Axitinib	0.943167	RTK signaling	11.3664	
3	SIDM00203	1021	Axitinib	3.024536	RTK signaling	8.0025	
4	SIDM01111	1021	Axitinib	0.640129	RTK signaling	7.3332	
...	...	...	...	...	...	...	
10292	SIDM00216	2040	Foretinib	-0.732719	RTK signaling	0.2510	
10293	SIDM00215	2040	Foretinib	1.027171	RTK signaling	0.0426	
10294	SIDM00193	2040	Foretinib	3.247533	RTK signaling	0.0000	
10295	SIDM00498	2040	Foretinib	1.633963	RTK signaling	1.6182	
10296	SIDM00049	2040	Foretinib	0.670544	RTK signaling	0.0426	
	KRT19	SPARC	RPL36A-HNRNPH2	S100P	...	TM4SF1-AS1	TP53BP1 \
0	2.6323	7.7819	9.1832	1.2079	...	0.0426	5.1803
1	0.0976	7.1986	1.8639	0.7137	...	0.2510	5.9174
2	0.4005	8.4297	2.4410	1.6088	...	0.0000	5.7390
3	0.1110	8.1244	1.6182	0.0976	...	0.0000	6.1108

4	2.0496	6.9437		9.7069	1.5110	...	0.0000	5.6499
...	...	...	...	...	...	...	...	...
10292	8.1185	0.0000		0.8237	10.3572	...	0.0976	4.0772
10293	9.3953	0.0144		0.3334	7.5042	...	0.2630	4.1367
10294	10.3296	0.1243		0.8156	8.8128	...	0.4114	4.4168
10295	11.6445	0.2750		1.3951	8.8762	...	0.8400	4.5429
10296	9.3408	0.0566		0.2016	9.1808	...	1.9069	4.5242

	CEP126	HIPK1	FEM1B	CYP4X1	RN7SL657P	MGRN1	RN7SL81P	IFI27L1
0	0.8237	4.7687	3.8738	0.0841	1.8319	4.5084	1.6276	4.3903
1	2.0250	6.5821	5.8610	0.6229	0.6135	5.5981	2.6253	4.7975
2	2.1342	5.0023	4.8268	0.4114	1.0635	5.5382	2.3618	4.7312
3	2.2357	5.6109	4.9977	0.5460	0.7137	5.1942	1.7181	4.4356
4	1.4906	5.4236	3.8166	0.0000	3.0925	4.5103	2.6253	5.1977
...	...	...	...	...	...	...	...	...
10292	1.4699	4.0170	4.5601	0.4647	0.0000	4.2149	2.0036	5.1260
10293	0.3674	3.9288	3.8510	1.0909	0.0000	4.6662	1.1953	4.4303
10294	0.7485	4.5078	3.9355	0.3448	0.0000	5.1562	0.8480	5.2854
10295	0.8639	4.3176	4.1018	0.5261	0.0000	4.7225	1.1570	4.1235
10296	0.9411	3.2824	3.9411	0.0976	0.0000	4.4370	0.7137	3.9392

[10297 rows x 10007 columns]

```
[52]: Merged_Data.shape
# Check dimensions
```

```
[52]: (10297, 10007)
```

```
[ ]: Merged_Data.to_csv("Merge_GDSC_Expression_IC50.csv", index=False)
# Save merged dataset
```

Data Preprocessing and Cleaning Shahad Alduwayrij

```
[50]: Processed_Data = pd.read_csv("Merge_GDSC_Expression_IC50.csv")
Processed_Data.head()
```

```
[50]: SANGER_MODEL_ID  DRUG_ID  DRUG_NAME  LN_IC50  PATHWAY_NAME  VIM  \
0      SIDM00848      1021  Axitinib  1.180880  RTK signaling  10.5437
1      SIDM00263      1021  Axitinib  0.767377  RTK signaling  7.3118
2      SIDM00269      1021  Axitinib  0.943167  RTK signaling  11.3664
3      SIDM00203      1021  Axitinib  3.024536  RTK signaling  8.0025
4      SIDM01111      1021  Axitinib  0.640129  RTK signaling  7.3332

      KRT19  SPARC  RPL36A-HNRNP2  S100P  ...  TM4SF1-AS1  TP53BP1  CEP126  \
0  2.6323  7.7819      9.1832  1.2079  ...      0.0426  5.1803  0.8237
1  0.0976  7.1986      1.8639  0.7137  ...      0.2510  5.9174  2.0250
2  0.4005  8.4297      2.4410  1.6088  ...      0.0000  5.7390  2.1342
```

```

3  0.1110  8.1244          1.6182  0.0976  ...      0.0000  6.1108  2.2357
4  2.0496  6.9437          9.7069  1.5110  ...      0.0000  5.6499  1.4906

```

```

      HIPK1  FEM1B  CYP4X1  RN7SL657P  MGRN1  RN7SL81P  IFI27L1
0  4.7687  3.8738  0.0841    1.8319  4.5084    1.6276  4.3903
1  6.5821  5.8610  0.6229    0.6135  5.5981    2.6253  4.7975
2  5.0023  4.8268  0.4114    1.0635  5.5382    2.3618  4.7312
3  5.6109  4.9977  0.5460    0.7137  5.1942    1.7181  4.4356
4  5.4236  3.8166  0.0000    3.0925  4.5103    2.6253  5.1977

```

[5 rows x 10007 columns]

```
[ ]: # Check dataset shape
Processed_Data.shape
```

```
[ ]: (10297, 10007)
```

```
[ ]: # Display dataset information
Processed_Data.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10297 entries, 0 to 10296
Columns: 10007 entries, SANGER_MODEL_ID to IFI27L1
dtypes: float64(10003), int64(1), object(3)
memory usage: 786.1+ MB

```

1.1 ##### Check Missing Values

```
[ ]: # Check which columns contain missing values (True = contains missing values)
print(Processed_Data.isnull().any().sort_values(ascending=False))
```

```

RNA5-8SN5      True
PRRC2B         True
F8A1           True
MAFIP          True
RNA5SP145      True
...
ARHGEF10L     False
EMSLR         False
SMAD6         False
PARP9         False
IFI27L1       False
Length: 10007, dtype: bool

```

```
[ ]: # Count missing values in each column and sort them from highest to lowest
missing_values = Processed_Data.isnull().sum().sort_values(ascending=False)
missing_values
```

```
[ ]: RNA5-8SN5      5954
      LUZP6         5954
      MAFIP         5954
      RNA5SP145     5954
      RNA5-8SN4     5954
      ...
      ARHGEF10L     0
      EMSLR         0
      SMAD6         0
      PARP9         0
      IFI27L1       0
      Length: 10007, dtype: int64
```

```
[ ]: # Display only columns that contain missing values
      Processed_Data.isnull().sum()[Processed_Data.isnull().sum() > 0]
```

```
[ ]: RNA5-8SN5      5954
      RNA5-8SN4     5954
      LUZP6         5954
      F8A1          4343
      F8A2          4343
      MAFIP         5954
      RNA5SP145     5954
      PRRC2B        4343
      dtype: int64
```

```
[ ]: # Calculate the total number of missing values in the dataset
      total_missing_values = Processed_Data.isnull().sum().sum()
      total_missing_values
```

```
[ ]: 42799
```

1.2 ##### Missing Value Handling

```
[ ]: # Calculate the percentage of missing values in each column
      missing_percentage = Processed_Data.isnull().mean() * 100
      missing_percentage.sort_values(ascending=False)
```

```
[ ]: RNA5-8SN5      57.822667
      LUZP6         57.822667
      MAFIP         57.822667
      RNA5SP145     57.822667
      RNA5-8SN4     57.822667
      ...
      ARHGEF10L     0.000000
      EMSLR         0.000000
      SMAD6         0.000000
```

```
PARP9          0.000000
IFI27L1        0.000000
Length: 10007, dtype: float64
```

```
[ ]: # Identify columns with more than 50% missing values
columns_to_drop = missing_percentage[missing_percentage > 50].index
columns_to_drop
```

```
[ ]: Index(['RNA5-8SN5', 'RNA5-8SN4', 'LUZP6', 'MAFIP', 'RNA5SP145'], dtype='object')
```

```
[ ]: # Drop columns that contain a high percentage of missing values
Processed_Data = Processed_Data.drop(columns=columns_to_drop)
```

```
[ ]: # Fill remaining missing numerical values using the median
# Median is used because it is less affected by extreme values
Processed_Data = Processed_Data.fillna(
    Processed_Data.median(numeric_only=True)
)
```

```
[ ]: # Check the total number of missing values after handling them
Processed_Data.isnull().sum().sum()
```

```
[ ]: 0
```

```
[ ]: # Check the final shape of the dataset after preprocessing
Processed_Data.shape
```

```
[ ]: (10297, 10002)
```

1.3 ##### Duplicate Checking

```
[ ]: # Check duplicate rows
Processed_Data.duplicated().sum()
```

```
[ ]: 0
```

1.4 ##### Data Type Validation

```
[ ]: # Display the data types of all columns in the dataset
# This step ensures that numerical features are stored in the correct format_
↳ for machine learning
Processed_Data.dtypes
```

```
[ ]: SANGER_MODEL_ID    object
     DRUG_ID          int64
     DRUG_NAME        object
     LN_IC50          float64
     PATHWAY_NAME      object
```

```

...
CYP4X1          float64
RN7SL657P       float64
MGRN1           float64
RN7SL81P        float64
IFI27L1         float64
Length: 10002, dtype: object

```

1.5 ##### Outlier Analysis

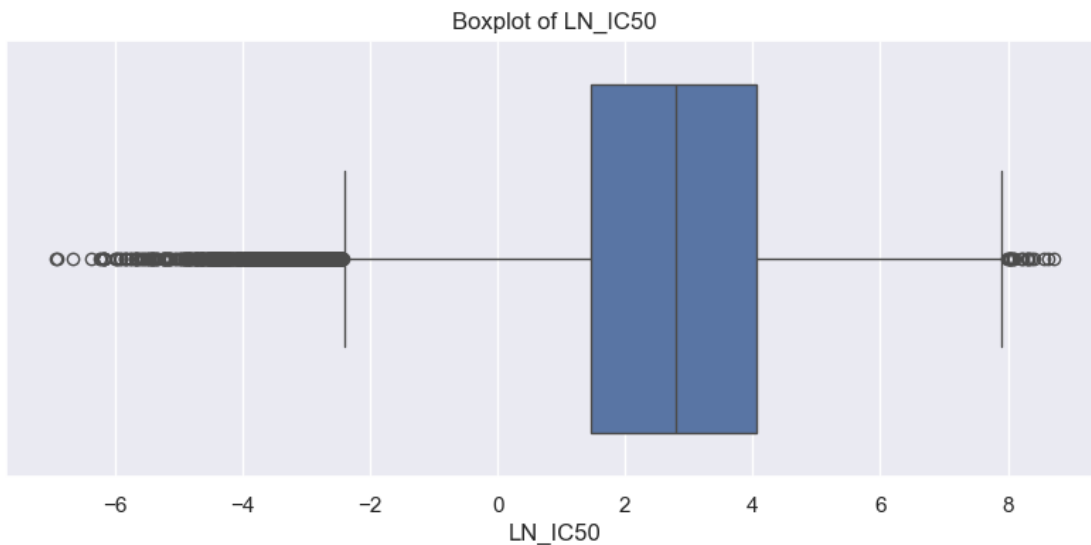
```

[ ]: # Visualize potential outliers in the target variable (LN_IC50) using a boxplot
plt.figure(figsize=(10,4))
sns.boxplot(x=Processed_Data["LN_IC50"])

plt.title("Boxplot of LN_IC50")
plt.xlabel("LN_IC50")

plt.show()

```



```

[ ]: # Detect potential outliers in the LN_IC50 target variable
# using the Interquartile Range (IQR) statistical method
Q1 = Processed_Data["LN_IC50"].quantile(0.25)
Q3 = Processed_Data["LN_IC50"].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

```

```
[ ]: # Extract samples with extreme LN_IC50 values
# that fall outside the normal IQR boundaries
outliers = Processed_Data[
    (Processed_Data["LN_IC50"] < lower_bound) |
    (Processed_Data["LN_IC50"] > upper_bound)
]

outliers.shape
```

```
[ ]: (605, 10002)
```

Outliers were intentionally retained because extreme drug response values may contain biologically significant information related to cancer sensitivity and resistance mechanisms

```
[ ]: # Select gene expression columns only by removing metadata columns
metadata_columns = [
    "SANGER_MODEL_ID",
    "DRUG_ID",
    "DRUG_NAME",
    "PATHWAY_NAME",
    "LN_IC50"
]

gene_columns = Processed_Data.columns.drop(metadata_columns)
```

```
[ ]: # Calculate summary statistics for all gene expression values
Processed_Data[gene_columns].describe()
```

```
[ ]:
```

	VIM	KRT19	SPARC	RPL36A-HNRNPH2	S100P \
count	10297.000000	10297.000000	10297.000000	10297.000000	10297.000000
mean	7.099701	4.963490	4.961530	5.709572	3.280831
std	4.037540	4.269523	4.136498	4.155548	3.546739
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	2.918400	1.056600	1.090900	0.933600	0.555800
50%	8.243200	3.102700	3.740900	8.453300	1.469900
75%	10.560000	9.393000	8.881800	9.350700	5.786600
max	14.440500	13.368000	14.310300	11.331600	14.264700

	C19orf33	FXVD3	UCHL1	LCN2	KRT8 \
count	10297.000000	10297.000000	10297.000000	10297.000000	10297.000000
mean	4.186892	3.177534	4.950605	2.680036	6.854004
std	3.831455	3.615186	3.873185	3.330942	3.843784
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.526100	0.286900	0.926000	0.263000	3.046100
50%	2.207900	0.903000	4.885100	1.014400	8.047300
75%	8.201700	6.790600	8.905000	4.468600	10.222400
max	11.047500	12.289100	11.206200	14.659400	13.537000

	...	TM4SF1-AS1	TP53BP1	CEP126	HIPK1	\
count	...	10297.000000	10297.000000	10297.000000	10297.000000	
mean	...	0.648914	4.685071	1.375377	4.143073	
std	...	0.810007	0.697040	0.934599	0.733998	
min	...	0.000000	0.000000	0.000000	0.000000	
25%	...	0.000000	4.273500	0.659900	3.684800	
50%	...	0.298700	4.727900	1.339100	4.101000	
75%	...	1.049600	5.138300	1.948600	4.588600	
max	...	4.880700	6.612900	5.497600	6.751400	

		FEM1B	CYP4X1	RN7SL657P	MGRN1	RN7SL81P	\
count	10297.000000	10297.000000	10297.000000	10297.000000	10297.000000	10297.000000	
mean		4.225093	0.450036	1.061239	4.785851	1.472955	
std		0.759074	0.847863	0.894208	0.715111	0.733713	
min		0.000000	0.000000	0.000000	0.000000	0.000000	
25%		3.734400	0.014400	0.201600	4.397100	0.963500	
50%		4.205500	0.070400	0.910700	4.849000	1.433000	
75%		4.718600	0.443600	1.731200	5.272800	1.996400	
max		7.365100	6.744600	3.861000	6.531800	3.860000	

	IFI27L1
count	10297.000000
mean	4.554973
std	0.839506
min	0.000000
25%	4.012600
50%	4.549100
75%	5.117300
max	6.957600

[8 rows x 9997 columns]

```
[ ]: # Use a sample of gene expression values to visualize outliers efficiently
gene_sample = Processed_Data[gene_columns].sample(
    n=40,
    axis=1,
    random_state=42
)

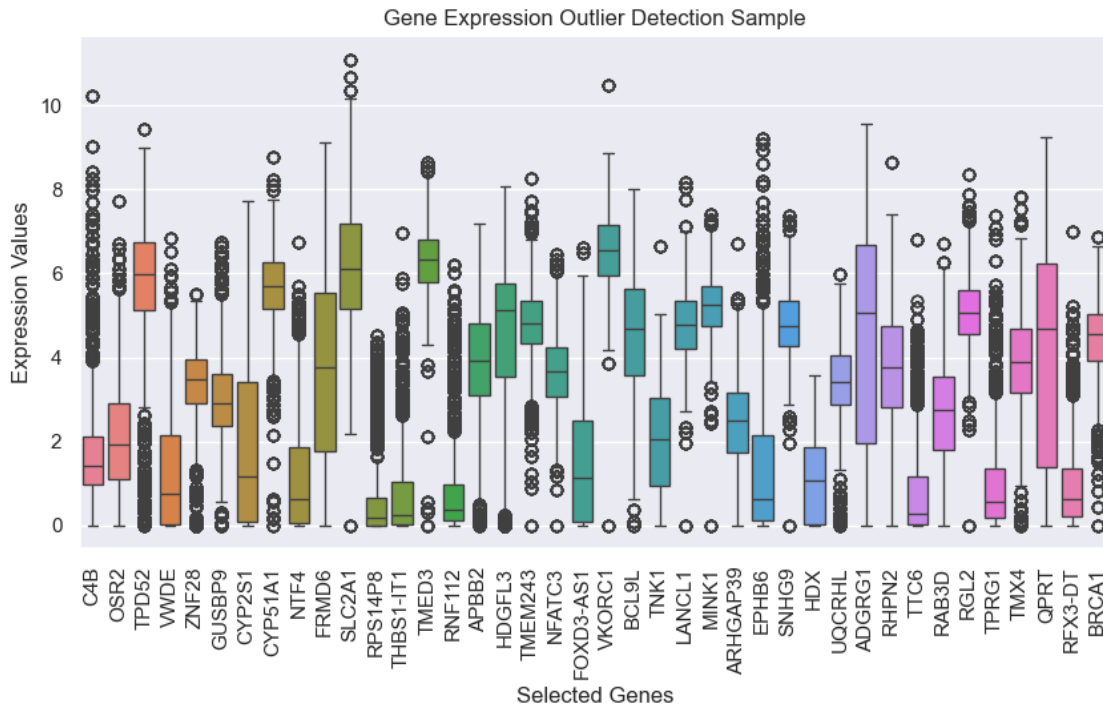
plt.figure(figsize=(10,5))

sns.boxplot(data=gene_sample)

plt.title("Gene Expression Outlier Detection Sample")
plt.xlabel("Selected Genes")
plt.ylabel("Expression Values")
```

```
plt.xticks(rotation=90)

plt.show()
```



Due to the high dimensionality of the dataset (~10,000 gene features), a random subset of genes was selected for outlier visualization to avoid memory and computational limitations.

1.6 ##### Final Dataset Preparation

```
[ ]: # Final check for missing values after preprocessing
Processed_Data.isnull().sum().sum()
```

```
[ ]: 0
```

```
[ ]: # Check the final shape of the processed dataset
Processed_Data.shape
```

```
[ ]: (10297, 10002)
```

```
[ ]: # Final check for duplicate rows
Processed_Data.duplicated().sum()
```

```
[ ]: 0
```

Exploratory Data Analysis (EDA) Renad Alsaud

The EDA process provided insights into gene expression variability, drug distribution, pathway frequency, and relationships between biological features and drug response values.

```
[ ]: # Display first 5 rows to understand dataset structure
Processed_Data.head()
```

```
[ ]: SANGER_MODEL_ID  DRUG_ID  DRUG_NAME  LN_IC50  PATHWAY_NAME  VIM  \
0      SIDM00848      1021  Axitinib  1.180880  RTK signaling  10.5437
1      SIDM00263      1021  Axitinib  0.767377  RTK signaling   7.3118
2      SIDM00269      1021  Axitinib  0.943167  RTK signaling  11.3664
3      SIDM00203      1021  Axitinib  3.024536  RTK signaling   8.0025
4      SIDM01111      1021  Axitinib  0.640129  RTK signaling   7.3332

      KRT19  SPARC  RPL36A-HNRNPH2  S100P  ...  TM4SF1-AS1  TP53BP1  CEP126  \
0  2.6323  7.7819                9.1832  1.2079  ...    0.0426   5.1803  0.8237
1  0.0976  7.1986                1.8639  0.7137  ...    0.2510   5.9174  2.0250
2  0.4005  8.4297                2.4410  1.6088  ...    0.0000   5.7390  2.1342
3  0.1110  8.1244                1.6182  0.0976  ...    0.0000   6.1108  2.2357
4  2.0496  6.9437                9.7069  1.5110  ...    0.0000   5.6499  1.4906

      HIPK1  FEM1B  CYP4X1  RN7SL657P  MGRN1  RN7SL81P  IFI27L1
0  4.7687  3.8738  0.0841    1.8319  4.5084    1.6276  4.3903
1  6.5821  5.8610  0.6229    0.6135  5.5981    2.6253  4.7975
2  5.0023  4.8268  0.4114    1.0635  5.5382    2.3618  4.7312
3  5.6109  4.9977  0.5460    0.7137  5.1942    1.7181  4.4356
4  5.4236  3.8166  0.0000    3.0925  4.5103    2.6253  5.1977
```

[5 rows x 10002 columns]

```
[ ]: # Show all column names
print(Processed_Data.columns)

# Check dataset shape (rows, columns)
Processed_Data.shape

# Get detailed information about data types and memory usage
Processed_Data.info()
```

```
Index(['SANGER_MODEL_ID', 'DRUG_ID', 'DRUG_NAME', 'LN_IC50', 'PATHWAY_NAME',
      'VIM', 'KRT19', 'SPARC', 'RPL36A-HNRNPH2', 'S100P',
      ...,
      'TM4SF1-AS1', 'TP53BP1', 'CEP126', 'HIPK1', 'FEM1B', 'CYP4X1',
      'RN7SL657P', 'MGRN1', 'RN7SL81P', 'IFI27L1'],
      dtype='object', length=10002)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10297 entries, 0 to 10296
Columns: 10002 entries, SANGER_MODEL_ID to IFI27L1
```

```
dtypes: float64(9998), int64(1), object(3)
memory usage: 785.8+ MB
```

```
[ ]: # Select only numeric columns (gene expression + IC50)
numeric_cols = Processed_Data.select_dtypes(include=['float64', 'int64']).
    ↪columns

# Remove DRUG_ID because it's just an identifier
numeric_cols = [col for col in numeric_cols if col != 'DRUG_ID']

# Display first few numeric columns
print(numeric_cols[:10])
```

```
['LN_IC50', 'VIM', 'KRT19', 'SPARC', 'RPL36A-HNRNPH2', 'S100P', 'C19orf33',
'FXYD3', 'UCHL1', 'LCN2']
```

```
[ ]: # Count how many times each drug appears in the dataset
drug_counts = Processed_Data['DRUG_NAME'].value_counts().head(10)

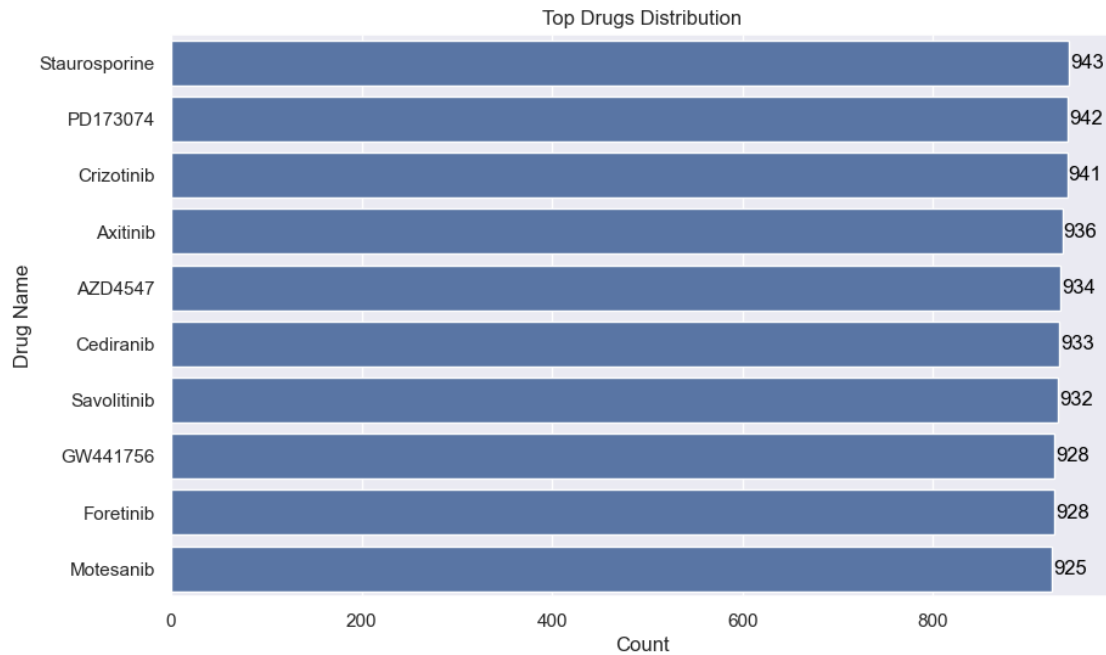
# Plot top drugs distribution
plt.figure(figsize=(10,6))

ax = sns.barplot(
    x=drug_counts.values,
    y=drug_counts.index
)

# Add count labels on bars
for i, v in enumerate(drug_counts.values):
    ax.text(v + 1, i, str(v), color='black', va='center')

plt.title("Top Drugs Distribution")
plt.xlabel("Count")
plt.ylabel("Drug Name")

plt.show()
```



```
[ ]: # Create distribution plots for selected genes/features
# KDE curve is added to better visualize the data distribution pattern
# This helps explore the distribution and variation of gene expression values
↳ and LN_IC50

cols = numeric_cols[:6]

plt.figure(figsize=(12,8))

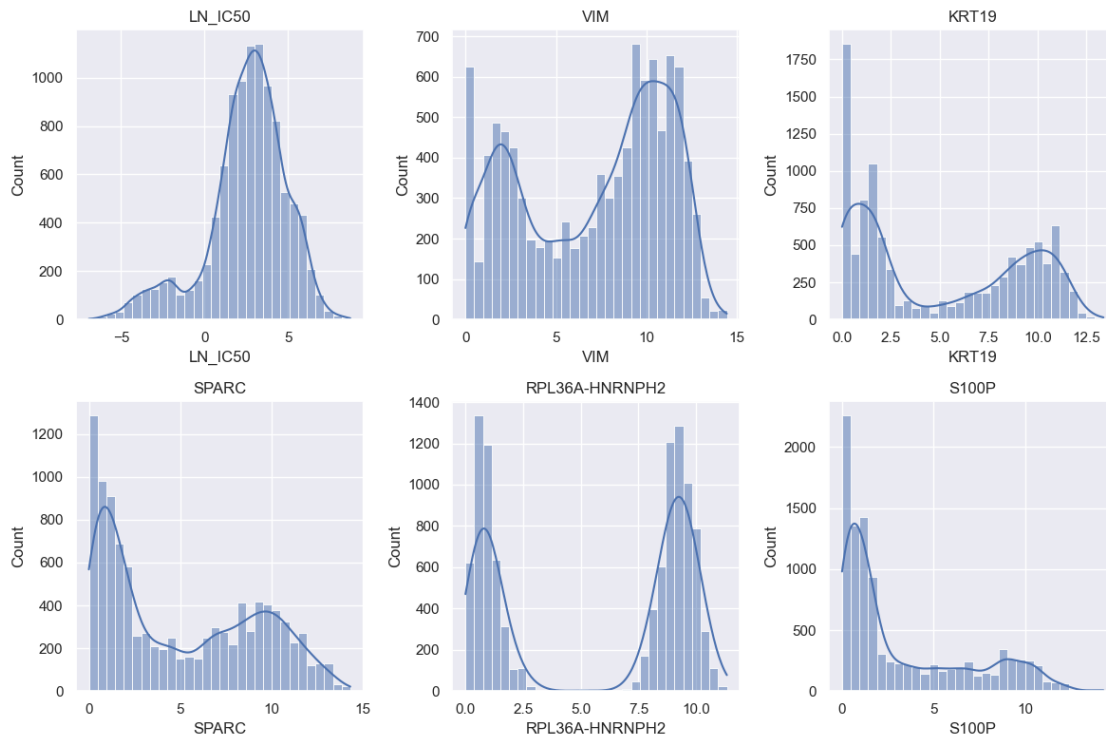
for i, col in enumerate(cols):
    plt.subplot(2,3,i+1)

    sns.histplot(Processed_Data[col], bins=30, kde=True)

    plt.title(col)

plt.tight_layout()

plt.show()
```



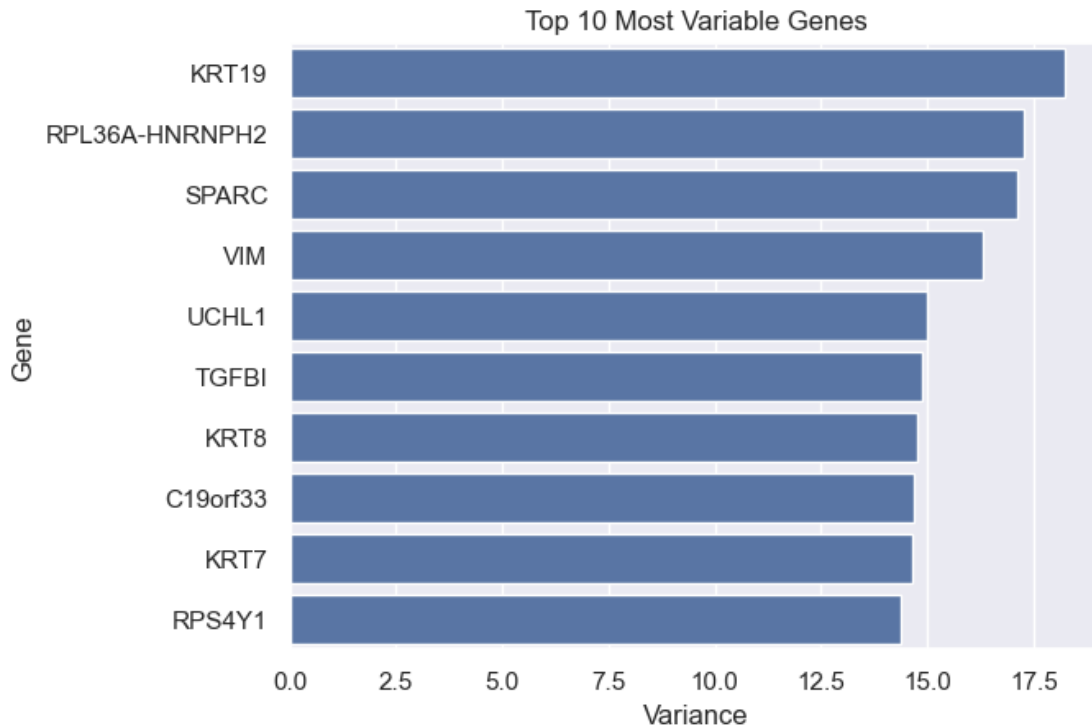
```
[ ]: # Define metadata and target columns
metadata_columns = ["SANGER_MODEL_ID", "DRUG_ID", "DRUG_NAME", "PATHWAY_NAME",
↪ "LN_IC50"]

[ ]: # Select only gene expression columns
gene_columns = [col for col in Processed_Data.columns if col not in
↪ metadata_columns]

[ ]: # Calculate variance for all gene expression features
gene_variance = Processed_Data[gene_columns].var()

[ ]: # Identify the top 10 most variable genes
top_variable_genes = gene_variance.sort_values(ascending=False).head(10)

[ ]: # Visualize the top 10 most variable genes
sns.barplot(x=top_variable_genes.values, y=top_variable_genes.index)
plt.title("Top 10 Most Variable Genes")
plt.xlabel("Variance")
plt.ylabel("Gene")
plt.show()
```



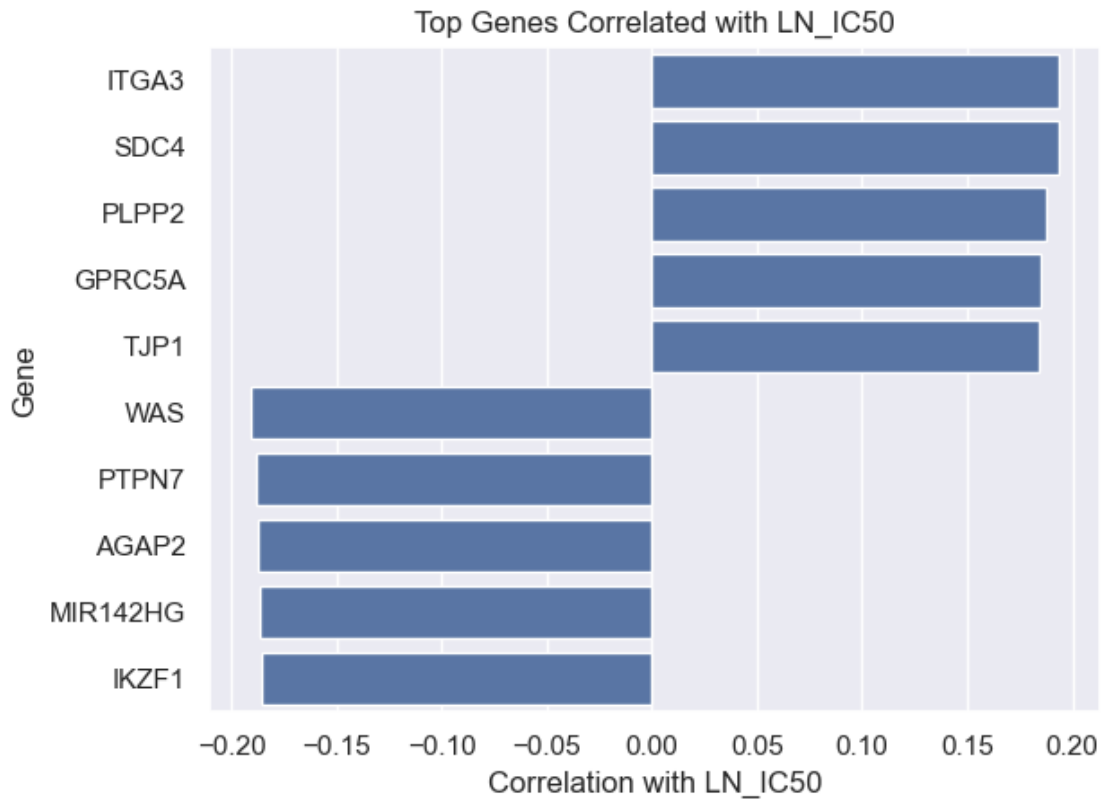
```
[ ]: # Calculate correlation between gene expression features and LN_IC50
gene_correlations = Processed_Data[gene_columns].
    ↪corrwith(Processed_Data["LN_IC50"])

[ ]: # Select the top positively correlated genes
top_positive_corr = gene_correlations.sort_values(ascending=False).head(5)

[ ]: # Select the top negatively correlated genes
top_negative_corr = gene_correlations.sort_values(ascending=True).head(5)

[ ]: # Combine positively and negatively correlated genes
top_corr_genes = pd.concat([top_positive_corr,top_negative_corr])

[ ]: # Visualize genes most correlated with LN_IC50
sns.barplot(x=top_corr_genes.values,y=top_corr_genes.index)
plt.title("Top Genes Correlated with LN_IC50")
plt.xlabel("Correlation with LN_IC50")
plt.ylabel("Gene")
plt.show()
```



Feature Engineering Reemas Alanzi

In this step, the cleaned genomic dataset (`Processed_Data`) was prepared for machine learning modeling.

The preprocessing workflow included: - separating genomic features from the target variable - selecting numerical features only - normalizing the data using `StandardScaler` - reducing dimensionality using PCA

The final output is a processed feature matrix ready for predictive modeling.

```
[ ]: # Display dataset shape
print("Processed Dataset Shape:")
print(Processed_Data.shape)
```

Processed Dataset Shape:
(10297, 10002)

```
[ ]: # Preview first rows

print("\nFirst 5 Rows of Dataset:")
display(Processed_Data.head())
```


First 5 Rows of Dataset:

	SANGER_MODEL_ID	DRUG_ID	DRUG_NAME	LN_IC50	PATHWAY_NAME	VIM	\
0	SIDM00848	1021	Axitinib	1.180880	RTK signaling	10.5437	
1	SIDM00263	1021	Axitinib	0.767377	RTK signaling	7.3118	
2	SIDM00269	1021	Axitinib	0.943167	RTK signaling	11.3664	
3	SIDM00203	1021	Axitinib	3.024536	RTK signaling	8.0025	
4	SIDM01111	1021	Axitinib	0.640129	RTK signaling	7.3332	

	KRT19	SPARC	RPL36A-HNRNPH2	S100P	...	TM4SF1-AS1	TP53BP1	CEP126	\
0	2.6323	7.7819	9.1832	1.2079	...	0.0426	5.1803	0.8237	
1	0.0976	7.1986	1.8639	0.7137	...	0.2510	5.9174	2.0250	
2	0.4005	8.4297	2.4410	1.6088	...	0.0000	5.7390	2.1342	
3	0.1110	8.1244	1.6182	0.0976	...	0.0000	6.1108	2.2357	
4	2.0496	6.9437	9.7069	1.5110	...	0.0000	5.6499	1.4906	

	HIPK1	FEM1B	CYP4X1	RN7SL657P	MGRN1	RN7SL81P	IFI27L1
0	4.7687	3.8738	0.0841	1.8319	4.5084	1.6276	4.3903
1	6.5821	5.8610	0.6229	0.6135	5.5981	2.6253	4.7975
2	5.0023	4.8268	0.4114	1.0635	5.5382	2.3618	4.7312
3	5.6109	4.9977	0.5460	0.7137	5.1942	1.7181	4.4356
4	5.4236	3.8166	0.0000	3.0925	4.5103	2.6253	5.1977

[5 rows x 10002 columns]

```
[ ]: # Separate LN_IC50 as the target variable
```

```
y = Processed_Data["LN_IC50"]
```

```
[ ]: # Display target variable information
```

```
print("Target Variable Shape:")
print(y.shape)

print("\nFirst 5 Values of LN_IC50:")
display(y.head())
```

Target Variable Shape:
(10297,)

First 5 Values of LN_IC50:

```
0    1.180880
1    0.767377
2    0.943167
3    3.024536
4    0.640129
Name: LN_IC50, dtype: float64
```

```
[ ]: # Remove target and identifier columns
```

```
X = Processed_Data.drop(
    columns=[
        "LN_IC50",
        "SANGER_MODEL_ID",
        "DRUG_NAME",
        "PATHWAY_NAME"
    ]
)
```

```
[ ]: # Convert DRUG_ID into categorical encoded features
```

```
drug_encoded = pd.get_dummies(X["DRUG_ID"].astype(str), prefix="DRUG").
    ↪astype("int8")
```

```
[ ]: # Remove original DRUG_ID column after encoding
```

```
X = X.drop(columns=["DRUG_ID"])
```

```
[ ]: # Combine original DRUG_ID column after encoding
```

```
X = pd.concat([X, drug_encoded], axis=1)
```

```
[ ]: # Check how many rows belong to each encoded drug
```

```
drug_encoded.sum().sort_values(ascending=False)
```

```
[ ]: DRUG_1034      943
      DRUG_1049      942
      DRUG_1083      941
      DRUG_1021      936
      DRUG_1786      934
      DRUG_1922      933
      DRUG_1936      932
      DRUG_1023      928
      DRUG_2040      928
      DRUG_1029      925
      DRUG_1463      698
      DRUG_1194      257
      dtype: int64
```

```
[ ]: # Display numerical feature information
```

```
print("Feature Matrix Shape:")
print(X.shape)

print("\nFirst 5 Rows of Features:")
display(X.head())
```

```
Feature Matrix Shape:
(10297, 10009)
```

First 5 Rows of Features:

	VIM	KRT19	SPARC	RPL36A-HNRNPH2	S100P	C19orf33	FXVD3	UCHL1	\
0	10.5437	2.6323	7.7819	9.1832	1.2079	0.5945	0.2869	9.2090	
1	7.3118	0.0976	7.1986	1.8639	0.7137	0.3103	0.1506	8.6297	
2	11.3664	0.4005	8.4297	2.4410	1.6088	0.0000	0.2141	10.1364	
3	8.0025	0.1110	8.1244	1.6182	0.0976	0.0000	0.0566	9.9343	
4	7.3332	2.0496	6.9437	9.7069	1.5110	0.9107	0.9184	9.9013	

	LCN2	KRT8	...	DRUG_1029	DRUG_1034	DRUG_1049	DRUG_1083	DRUG_1194	\
0	0.2630	5.7616	...	0	0	0	0	0	
1	0.0426	4.1260	...	0	0	0	0	0	
2	0.2630	1.1177	...	0	0	0	0	0	
3	0.0000	0.2987	...	0	0	0	0	0	
4	1.2987	2.5753	...	0	0	0	0	0	

	DRUG_1463	DRUG_1786	DRUG_1922	DRUG_1936	DRUG_2040
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

[5 rows x 10009 columns]

Feature Scaling Genomic features were normalized using `StandardScaler`.

This step standardizes feature values by transforming: - mean = 0 - standard deviation = 1

Normalization ensures that all genomic features contribute equally during machine learning training.

```
[ ]: # Seperate gene expression features
gene_features = X[gene_columns]
```

```
[ ]: # Standardize genomic features
scaler = StandardScaler()

gene_scaled = scaler.fit_transform(gene_features)
```

```
[ ]: # Convert scaled genes back into dataframe

gene_scaled = pd.DataFrame(
    gene_scaled,
    columns=gene_columns,
    index=X.index
)
```

```
[ ]: # Combine scaled genes with encoded drug columns
feature_matrix = pd.concat([gene_scaled, drug_encoded], axis=1)
```

```
[ ]: # Display normalized feature matrix

print("Normalized Feature Matrix Shape:")
print(feature_matrix.shape)

print("\nFirst 5 Rows of Normalized Features:")
display(feature_matrix.head())
```

Normalized Feature Matrix Shape:
(10297, 10009)

First 5 Rows of Normalized Features:

	VIM	KRT19	SPARC	RPL36A-HNRNPH2	S100P	C19orf33	FXD3	\
0	0.853036	-0.546034	0.681859	0.835942	-0.584490	-0.937651	-0.799620	
1	0.052534	-1.139735	0.540839	-0.925476	-0.723836	-1.011830	-0.837324	
2	1.056809	-1.068787	0.838472	-0.786594	-0.471451	-1.092821	-0.819758	
3	0.223612	-1.136597	0.764662	-0.984604	-0.897553	-1.092821	-0.863326	
4	0.057835	-0.682519	0.479214	0.961972	-0.499027	-0.855119	-0.624932	

	UCHL1	LCN2	KRT8	...	DRUG_1029	DRUG_1034	DRUG_1049	\
0	1.099509	-0.725667	-0.284214	...	0	0	0	
1	0.949935	-0.791837	-0.709753	...	0	0	0	
2	1.338962	-0.725667	-1.492431	...	0	0	0	
3	1.286780	-0.804627	-1.705512	...	0	0	0	
4	1.278260	-0.414718	-1.113203	...	0	0	0	

	DRUG_1083	DRUG_1194	DRUG_1463	DRUG_1786	DRUG_1922	DRUG_1936	DRUG_2040
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0

[5 rows x 10009 columns]

```
[ ]: # Add LN_IC50 back to the dataset

feature_matrix["LN_IC50"] = y.values
```

```
[ ]: # Display final organized feature matrix

print("Final Feature Matrix Shape:")
print(feature_matrix.shape)
```

```
print("\nFirst 5 Rows of Final Feature Matrix:")
display(feature_matrix.head())
```

Final Feature Matrix Shape:
(10297, 10010)

First 5 Rows of Final Feature Matrix:

	VIM	KRT19	SPARC	RPL36A-HNRNPH2	S100P	C19orf33	FXD3	\
0	0.853036	-0.546034	0.681859	0.835942	-0.584490	-0.937651	-0.799620	
1	0.052534	-1.139735	0.540839	-0.925476	-0.723836	-1.011830	-0.837324	
2	1.056809	-1.068787	0.838472	-0.786594	-0.471451	-1.092821	-0.819758	
3	0.223612	-1.136597	0.764662	-0.984604	-0.897553	-1.092821	-0.863326	
4	0.057835	-0.682519	0.479214	0.961972	-0.499027	-0.855119	-0.624932	

	UCHL1	LCN2	KRT8	...	DRUG_1034	DRUG_1049	DRUG_1083	\
0	1.099509	-0.725667	-0.284214	...	0	0	0	
1	0.949935	-0.791837	-0.709753	...	0	0	0	
2	1.338962	-0.725667	-1.492431	...	0	0	0	
3	1.286780	-0.804627	-1.705512	...	0	0	0	
4	1.278260	-0.414718	-1.113203	...	0	0	0	

	DRUG_1194	DRUG_1463	DRUG_1786	DRUG_1922	DRUG_1936	DRUG_2040	LN_IC50
0	0	0	0	0	0	0	1.180880
1	0	0	0	0	0	0	0.767377
2	0	0	0	0	0	0	0.943167
3	0	0	0	0	0	0	3.024536
4	0	0	0	0	0	0	0.640129

[5 rows x 10010 columns]

```
[ ]: # Save final processed genomic feature matrix

feature_matrix.to_csv(
    "The_Normalized_Genomic_Feature_Matrix.csv",
    index=False
)
```

Output The final processed dataset was saved as:

The_Normalized_Genomic_Feature_Matrix.csv

The feature matrix is now ready for: - machine learning training - predictive modeling - drug response prediction - statistical analysis

Machine Learning Model Development Lamees Yasser (Model Building and the models)

```
[ ]: # XGBoost import
# If this gives error, run: !pip install xgboost
```

```
try:
    from xgboost import XGBRegressor
    print("XGBoost imported successfully.")
except ModuleNotFoundError:
    XGBRegressor = None
    print("XGBoost is not installed.")
    print("Run this in a separate cell:")
    print("!pip install xgboost")
```

XGBoost imported successfully.

```
[22]: # 2. Load Dataset
df_genomic = pd.read_csv("The_Normalized_Genomic_Feature_Matrix.csv")

print("Data loaded successfully.")
print("Shape:", df_genomic.shape)
print("Last 5 columns:")
print(df_genomic.columns[-5:])
```

Data loaded successfully.

Shape: (10297, 10010)

Last 5 columns:

Index(['DRUG_1786', 'DRUG_1922', 'DRUG_1936', 'DRUG_2040', 'LN_IC50'],
dtype='object')

```
[ ]: # =====
# 3. Basic Data Cleaning
# =====

# Remove rows with missing values
df_genomic = df_genomic.dropna()

print("Shape after dropping missing values:", df_genomic.shape)
```

Shape after dropping missing values: (10297, 10010)

```
[24]: # =====
# 4. Define Features and Target
# =====

X = df_genomic.drop("LN_IC50", axis=1)
y = df_genomic["LN_IC50"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (10297, 10009)

y shape: (10297,)

```
[ ]: # =====
# 5. Train/Test Split
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (8237, 10009)
X_test shape: (2060, 10009)
y_train shape: (8237,)
y_test shape: (2060,)
```

```
[ ]: # =====
# 6. Linear Regression Baseline
# =====
# Linear Regression is used as a simple baseline model.
# It does not need hyperparameter tuning.

linear_model = LinearRegression()

# Train
linear_model.fit(X_train, y_train)

# Predict
linear_pred = linear_model.predict(X_test)

# Evaluate
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_pred))
linear_mae = mean_absolute_error(y_test, linear_pred)
linear_r2 = r2_score(y_test, linear_pred)

print("Linear Regression Results")
print("-----")
print("RMSE:", linear_rmse)
print("MAE:", linear_mae)
print("R2:", linear_r2)
```

```
Linear Regression Results
-----
```

RMSE: 1.0275931515879517
MAE: 0.7775671807025354
R2: 0.8200327192484178

Linear Regression was used as a baseline model. It achieved an R^2 score of 0.82, indicating good baseline predictive performance.

```
[ ]: # =====  
# 7. Ridge Regression with Light Hyperparameter Tuning  
# =====  
# Ridge is a regularized linear model.  
# It is more stable than ordinary Linear Regression for high-dimensional data.  
  
ridge_model = Ridge()  
  
ridge_params = {  
    "alpha": [0.1, 1, 10, 100, 1000]  
}  
  
ridge_search = RandomizedSearchCV(  
    estimator=ridge_model,  
    param_distributions=ridge_params,  
    n_iter=5,  
    cv=3,  
    scoring="neg_root_mean_squared_error",  
    random_state=42,  
    n_jobs=1  
)  
  
# Train with light tuning  
ridge_search.fit(X_train, y_train)  
  
# Best Ridge model  
best_ridge_model = ridge_search.best_estimator_  
  
# Predict  
ridge_pred = best_ridge_model.predict(X_test)  
  
# Evaluate  
ridge_rmse = np.sqrt(mean_squared_error(y_test, ridge_pred))  
ridge_mae = mean_absolute_error(y_test, ridge_pred)  
ridge_r2 = r2_score(y_test, ridge_pred)  
  
print("Ridge Regression Results")  
print("-----")  
print("Best Parameters:", ridge_search.best_params_)  
print("Best CV RMSE:", -ridge_search.best_score_)  
print("Test RMSE:", ridge_rmse)
```



```
print("Test MAE:", ridge_mae)
print("Test R2:", ridge_r2)
```

Ridge Regression Results

```
-----
Best Parameters: {'alpha': 0.1}
Best CV RMSE: 1.1097465142371385
Test RMSE: 1.0261019325247849
Test MAE: 0.7755428715642161
Test R2: 0.8205546688406311
```

Ridge Regression was used as a regularized linear model. The alpha parameter was tuned using RandomizedSearchCV. The best alpha was 0.1, and the model slightly improved performance compared with ordinary Linear Regression.

```
[ ]: # =====
# 8. Random Forest with Very Light RandomizedSearchCV
# =====
# Random Forest is computationally heavy for this dataset.
# Therefore, we use a very small randomized search.

rf_model = RandomForestRegressor(
    random_state=42,
    n_jobs=1
)

rf_params = {
    "n_estimators": [20, 50],
    "max_depth": [5, 10],
    "min_samples_leaf": [2, 4],
    "max_features": ["sqrt"]
}

rf_search = RandomizedSearchCV(
    estimator=rf_model,
    param_distributions=rf_params,
    n_iter=5,
    cv=3,
    scoring="neg_root_mean_squared_error",
    random_state=42,
    n_jobs=1,
    verbose=2
)

# Train with very light tuning
rf_search.fit(X_train, y_train)

# Best Random Forest model
```

```

best_rf_model = rf_search.best_estimator_

# Predict
rf_pred = best_rf_model.predict(X_test)

# Evaluate
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_r2 = r2_score(y_test, rf_pred)

print("Random Forest Results")
print("-----")
print("Best Parameters:", rf_search.best_params_)
print("Best CV RMSE:", -rf_search.best_score_)
print("Test RMSE:", rf_rmse)
print("Test MAE:", rf_mae)
print("Test R2:", rf_r2)

```

```

Fitting 3 folds for each of 5 candidates, totalling 15 fits
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=2, n_estimators=50;
total time= 6.5s
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=2, n_estimators=50;
total time= 6.6s
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=2, n_estimators=50;
total time= 6.5s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=2, n_estimators=50;
total time= 14.0s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=2, n_estimators=50;
total time= 13.3s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=2, n_estimators=50;
total time= 14.0s
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=2, n_estimators=20;
total time= 2.9s
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=2, n_estimators=20;
total time= 2.8s
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=2, n_estimators=20;
total time= 3.0s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=4, n_estimators=50;
total time= 11.8s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=4, n_estimators=50;
total time= 11.2s
[CV] END max_depth=10, max_features=sqrt, min_samples_leaf=4, n_estimators=50;
total time= 11.2s
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=4, n_estimators=20;
total time= 3.0s
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=4, n_estimators=20;
total time= 3.0s
[CV] END max_depth=5, max_features=sqrt, min_samples_leaf=4, n_estimators=20;

```

```
total time= 3.2s
Random Forest Results
```

```
-----
Best Parameters: {'n_estimators': 50, 'min_samples_leaf': 2, 'max_features':
'sqrt', 'max_depth': 10}
Best CV RMSE: 2.21801799070079
Test RMSE: 2.208211891343591
Test MAE: 1.6130302380284822
Test R2: 0.16893950051762874
```

Random Forest showed weaker performance compared with the linear and boosting models. This may be due to the high dimensionality of the genomic feature matrix and the lightweight hyperparameter search used to reduce computational cost.

```
[ ]: # =====
# 9. XGBoost with Very Light RandomizedSearchCV
# =====
# XGBoost is usually more efficient than Random Forest for this type of data.
# We still keep the search very small to avoid freezing the laptop.

if XGBRegressor is not None:

    xgb_model = XGBRegressor(
        objective="reg:squarederror",
        random_state=42,
        n_jobs=1
    )

    xgb_params = {
        "n_estimators": [50, 100],
        "max_depth": [2, 3],
        "learning_rate": [0.05, 0.1],
        "subsample": [0.8],
        "colsample_bytree": [0.8]
    }

    xgb_search = RandomizedSearchCV(
        estimator=xgb_model,
        param_distributions=xgb_params,
        n_iter=5,
        cv=3,
        scoring="neg_root_mean_squared_error",
        random_state=42,
        n_jobs=1,
        verbose=2
    )

    # Train with very light tuning
```

```

xgb_search.fit(X_train, y_train)

# Best XGBoost model
best_xgb_model = xgb_search.best_estimator_

# Predict
xgb_pred = best_xgb_model.predict(X_test)

# Evaluate
xgb_rmse = np.sqrt(mean_squared_error(y_test, xgb_pred))
xgb_mae = mean_absolute_error(y_test, xgb_pred)
xgb_r2 = r2_score(y_test, xgb_pred)

print("XGBoost Results")
print("-----")
print("Best Parameters:", xgb_search.best_params_)
print("Best CV RMSE:", -xgb_search.best_score_)
print("Test RMSE:", xgb_rmse)
print("Test MAE:", xgb_mae)
print("Test R2:", xgb_r2)

else:
    xgb_rmse = None
    xgb_mae = None
    xgb_r2 = None
    best_xgb_model = None

print("XGBoost was skipped because it is not installed.")

```

Fitting 3 folds for each of 5 candidates, totalling 15 fits

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=2, n_estimators=100, subsample=0.8; total time= 1.9min

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=2, n_estimators=100, subsample=0.8; total time= 1.7min

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=2, n_estimators=100, subsample=0.8; total time= 1.7min

[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=2, n_estimators=100, subsample=0.8; total time= 1.7min

[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=2, n_estimators=100, subsample=0.8; total time= 1.6min

[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=2, n_estimators=100, subsample=0.8; total time= 1.7min

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=2, n_estimators=50, subsample=0.8; total time= 1.0min

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=2, n_estimators=50, subsample=0.8; total time= 1.0min

[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=2, n_estimators=50, subsample=0.8; total time= 1.0min

```
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3, n_estimators=100,
subsample=0.8; total time= 2.4min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3, n_estimators=100,
subsample=0.8; total time= 2.2min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3, n_estimators=100,
subsample=0.8; total time= 1.9min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3, n_estimators=50,
subsample=0.8; total time= 1.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3, n_estimators=50,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3, n_estimators=50,
subsample=0.8; total time= 1.2min
XGBoost Results
-----
```

```
Best Parameters: {'subsample': 0.8, 'n_estimators': 100, 'max_depth': 3,
'learning_rate': 0.1, 'colsample_bytree': 0.8}
Best CV RMSE: 1.0898000339948017
Test RMSE: 1.0528725236801415
Test MAE: 0.8043593125758561
Test R2: 0.8110692116796967
```

XGBoost achieved good predictive performance with an R^2 of 0.811, but Ridge Regression achieved slightly lower RMSE and higher R^2 on the test set.

```
[ ]: # =====
# 10. Final Model Comparison Table
# =====

results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Ridge Regression",
        "Random Forest",
        "XGBoost"
    ],
    "Test RMSE": [
        linear_rmse,
        ridge_rmse,
        rf_rmse,
        xgb_rmse
    ],
    "Test MAE": [
        linear_mae,
        ridge_mae,
        rf_mae,
        xgb_mae
    ],
    "Test R2": [
```

```

        linear_r2,
        ridge_r2,
        rf_r2,
        xgb_r2
    ]
})

results

```

```

[ ]:
      Model  Test RMSE  Test MAE  Test R2
0  Linear Regression    1.027593  0.777567  0.820033
1   Ridge Regression    1.026102  0.775543  0.820555
2   Random Forest     2.208212  1.613030  0.168940
3      XGBoost         1.052873  0.804359  0.811069

```

The models were compared using RMSE, MAE, and R^2 . Ridge Regression achieved the best overall performance, with the lowest RMSE and highest R^2 score.

```

[ ]: # =====
# 11. Select Best Model Based on Test RMSE
# =====
# Lower RMSE means better prediction performance.

results_without_missing = results.dropna()

best_model_result = results_without_missing.loc[
    results_without_missing["Test RMSE"].idxmin()
]

print("Best Model Based on Test RMSE")
print("-----")
print(best_model_result)

```

Best Model Based on Test RMSE

```

-----
Model          Ridge Regression
Test RMSE              1.026102
Test MAE               0.775543
Test R2               0.820555
Name: 1, dtype: object

```

Based on the test RMSE, Ridge Regression was selected as the best-performing model. It achieved the lowest RMSE of 1.026 and the highest R^2 score of 0.821 among the tested models.

```

[ ]: # =====
# 12. Save Results and Models
# =====
# This section is useful for handover.

```

```

import joblib

# Save results table
results.to_csv("model_comparison_results.csv", index=False)

# Save trained models
joblib.dump(linear_model, "linear_regression_model.pkl")
joblib.dump(best_ridge_model, "ridge_regression_model.pkl")
joblib.dump(best_rf_model, "random_forest_model.pkl")

if best_xgb_model is not None:
    joblib.dump(best_xgb_model, "xgboost_model.pkl")

# Save feature names
joblib.dump(list(X.columns), "feature_names.pkl")

print("Results and models saved successfully.")

```

Results and models saved successfully.

Model Evaluation and Validation Raghad Althobaiti

```

[5]: sns.set_theme(style="whitegrid")

results_df = pd.read_csv(
    "model_comparison_results.csv"
)

results_df

```

```

[5]:

```

	Model	Test RMSE	Test MAE	Test R2
0	Linear Regression	1.027593	0.777567	0.820033
1	Ridge Regression	1.026102	0.775543	0.820555
2	Random Forest	2.208212	1.613030	0.168940
3	XGBoost	1.052873	0.804359	0.811069

```

[ ]: print(results_df.columns)

```

```
Index(['Model', 'Test RMSE', 'Test MAE', 'Test R2'], dtype='object')
```

```

[ ]: plt.figure(figsize=(10,6))

sns.barplot(
    data=results_df,
    x="Model",
    y="Test R2"
)

```

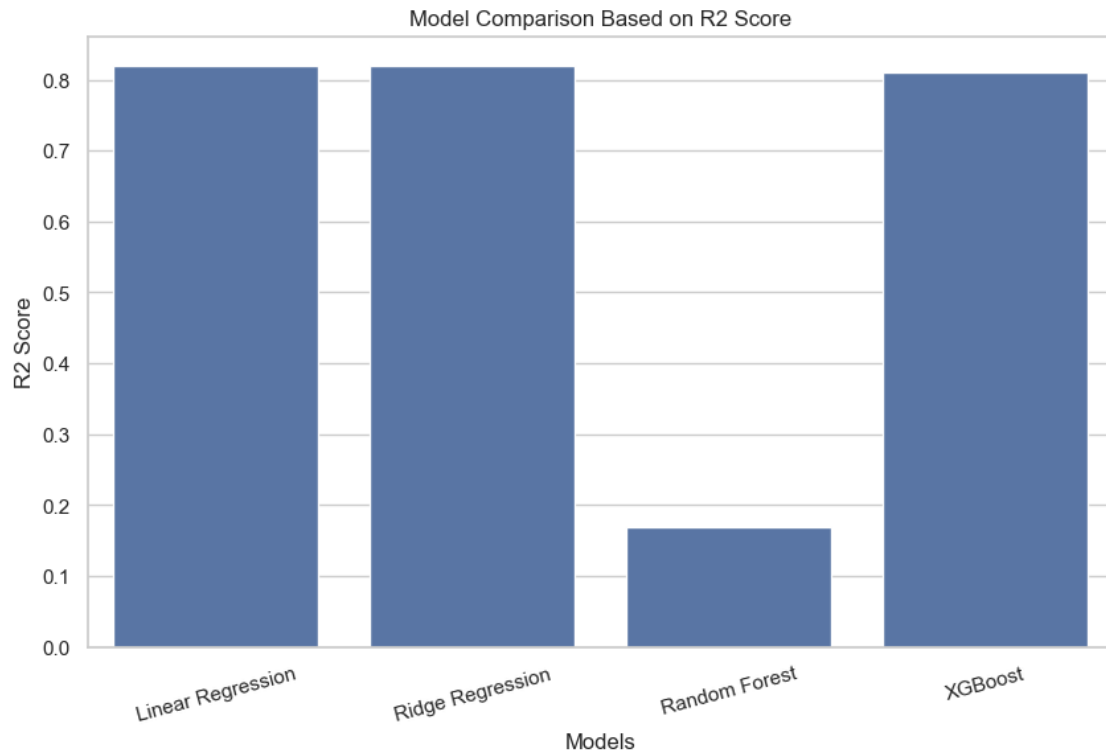
```
plt.title("Model Comparison Based on R2 Score")

plt.xlabel("Models")

plt.ylabel("R2 Score")

plt.xticks(rotation=15)

plt.show()
```



```
[ ]: plt.figure(figsize=(10,6))

sns.barplot(
    data=results_df,
    x="Model",
    y="Test RMSE"
)

plt.title("Model Comparison Based on RMSE")

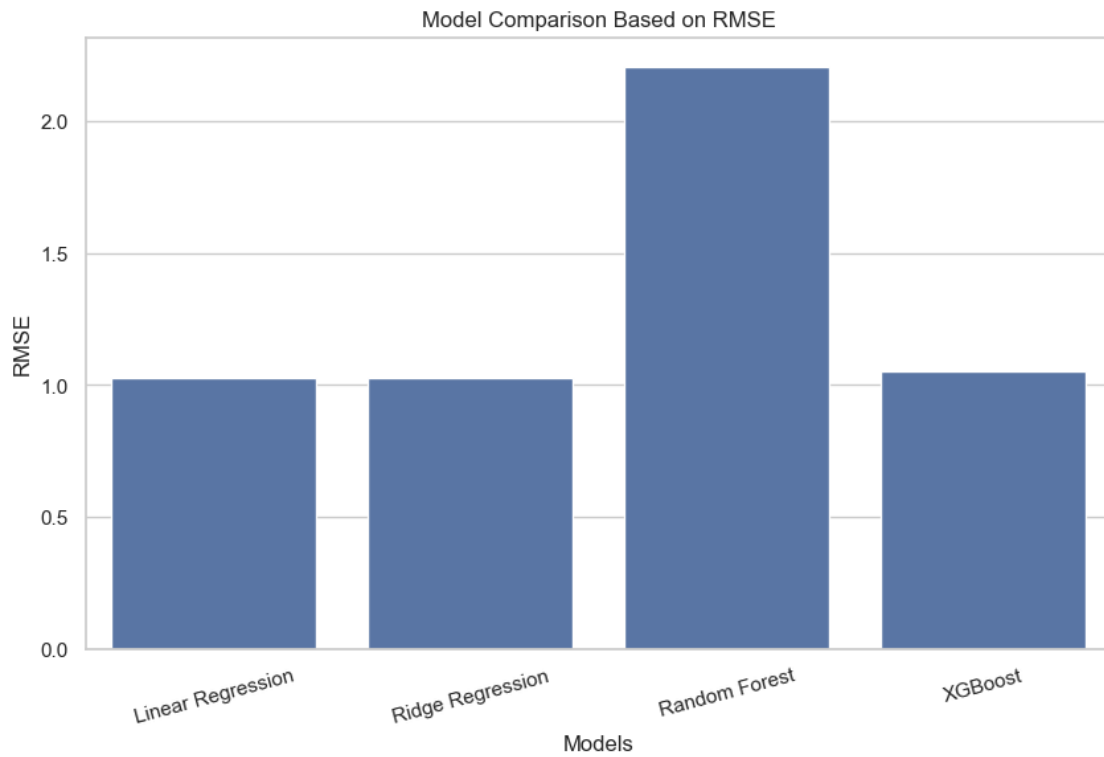
plt.xlabel("Models")
```



```
plt.ylabel("RMSE")

plt.xticks(rotation=15)

plt.show()
```



```
[ ]: plt.figure(figsize=(10,6))

sns.barplot(
    data=results_df,
    x="Model",
    y="Test MAE"
)

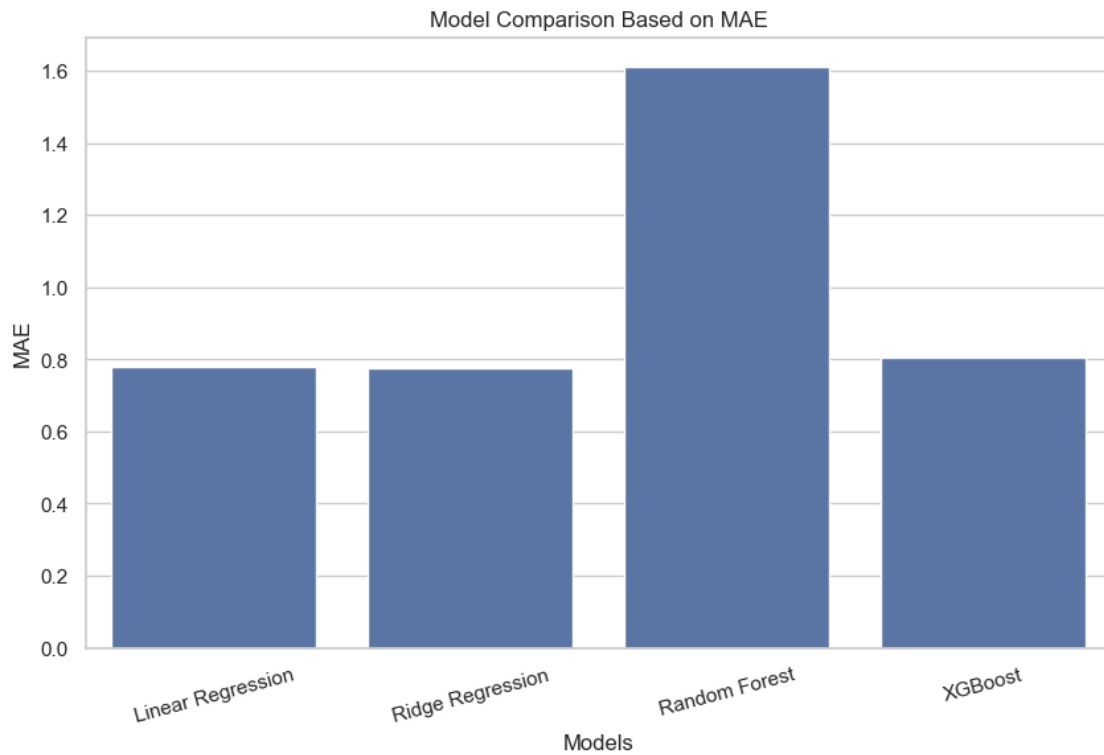
plt.title("Model Comparison Based on MAE")

plt.xlabel("Models")

plt.ylabel("MAE")

plt.xticks(rotation=15)
```

```
plt.show()
```



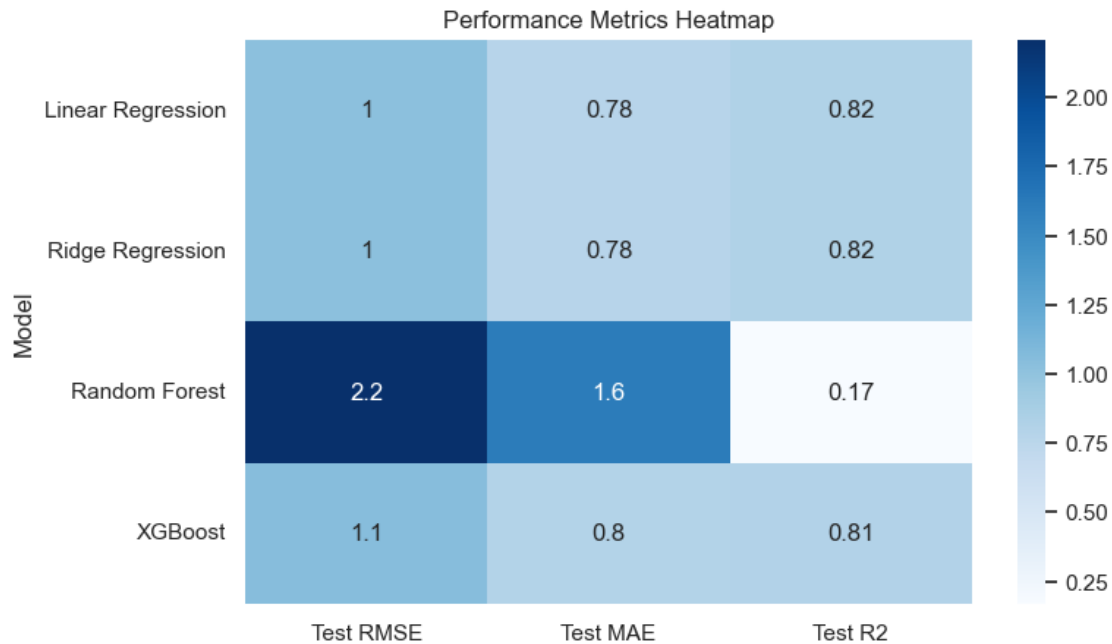
```
[ ]: plt.figure(figsize=(8,5))

heatmap_data = results_df.set_index("Model")

sns.heatmap(
    heatmap_data,
    annot=True,
    cmap="Blues"
)

plt.title("Performance Metrics Heatmap")

plt.show()
```



```
[ ]: ridge_model = joblib.load("ridge_regression_model.pkl")
feature_names = joblib.load("feature_names.pkl")

print("Model and features loaded successfully")
print("Number of features:", len(feature_names))
```

Model and features loaded successfully
Number of features: 10009

C:\Users\Ragha\anaconda3\Lib\site-packages\sklearn\base.py:376:
InconsistentVersionWarning: Trying to unpickle estimator Ridge from version 1.5.1 when using version 1.4.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(

```
[ ]: external_expression = pd.read_csv(
    "external_expression.csv.csv"
)

external_response = pd.read_csv(
    "external_response.csv.csv"
)

print(external_expression.shape)
```

```
print(external_response.shape)
```

C:\Users\Ragha\AppData\Local\Temp\ipykernel_8712\2342827489.py:5: DtypeWarning: Columns (16,17) have mixed types. Specify dtype option on import or set low_memory=False.

```
external_response = pd.read_csv(  
(1684, 19206)  
(753778, 20)
```

```
[ ]: print(external_expression.columns.tolist()[:30])
```

```
print(external_response.columns.tolist()[:30])
```

```
['Unnamed: 0', 'TSPAN6 (7105)', 'TNMD (64102)', 'DPM1 (8813)', 'SCYL3 (57147)',  
'FIRRM (55732)', 'FGR (2268)', 'CFH (3075)', 'FUCA2 (2519)', 'GCLC (2729)',  
'NFYA (4800)', 'STPG1 (90529)', 'NIPAL3 (57185)', 'LAS1L (81887)', 'ENPP4  
(22875)', 'SEMA3F (6405)', 'CFTR (1080)', 'ANKIB1 (54467)', 'CYP51A1 (1595)',  
'KRIT1 (889)', 'RAD52 (5893)', 'BAD (572)', 'LAP3 (51056)', 'CD99 (4267)',  
'HS3ST1 (9957)', 'AOC1 (26)', 'WNT16 (51384)', 'HECW1 (23072)', 'MAD1L1 (8379)',  
'LASP1 (3927)']  
['broad_id', 'depmap_id', 'ccle_name', 'screen_id', 'upper_limit',  
'lower_limit', 'slope', 'r2', 'auc', 'ec50', 'ic50', 'passed_str_profiling',  
'row_name', 'name', 'moa', 'target', 'disease.area', 'indication', 'smiles',  
'phase']
```

```
[ ]: external_expression = external_expression.drop(  
      columns=["Unnamed: 0"]  
)
```

```
print(external_expression.shape)
```

```
(1684, 19205)
```

```
[ ]: print(len(feature_names))
```

```
print(external_expression.shape)
```

```
10009
```

```
(1684, 19205)
```

```
[ ]: X_external = external_expression.copy()
```

```
missing_cols = [  
    col for col in feature_names  
    if col not in X_external.columns  
)
```

```
missing_df = pd.DataFrame(  
)
```

```

    0,
    index=X_external.index,
    columns=missing_cols
)

X_external = pd.concat(
    [X_external, missing_df],
    axis=1
)

X_external = X_external[feature_names]

print(X_external.shape)

```

(1684, 10009)

```

[ ]: external_predictions = ridge_model.predict(X_external)

print(external_predictions[:20])

```

```

[2.59884269 2.59884269 2.59884269 2.59884269 2.59884269 2.59884269
 2.59884269 2.59884269 2.59884269 2.59884269 2.59884269 2.59884269
 2.59884269 2.59884269 2.59884269 2.59884269 2.59884269 2.59884269
 2.59884269 2.59884269]

```

```

[ ]: matched_features = [
    col for col in feature_names
    if col in external_expression.columns
]

print("Matched features:", len(matched_features))
print(matched_features[:20])

```

Matched features: 0
[]

```

[ ]: external_expression_clean = external_expression.copy()

external_expression_clean.columns = (
    external_expression_clean.columns
    .str.replace(r"\s*(\d+)\s*", "", regex=True)
)

print(external_expression_clean.columns[:20])

```

```

Index(['TSPAN6', 'TNMD', 'DPM1', 'SCYL3', 'FIRM', 'FGR', 'CFH', 'FUCA2',
      'GCLC', 'NFYA', 'STPG1', 'NIPAL3', 'LAS1L', 'ENPP4', 'SEMA3F', 'CFTR',
      'ANKIB1', 'CYP51A1', 'KRIT1', 'RAD52'],
      dtype='object')

```

```
[ ]: matched_features = [
    col for col in feature_names
    if col in external_expression_clean.columns
]

print("Matched features after cleaning:", len(matched_features))
print(matched_features[:20])
```

Matched features after cleaning: 8654

```
['VIM', 'KRT19', 'SPARC', 'S100P', 'C19orf33', 'FXVD3', 'UCHL1', 'LCN2', 'KRT8',
'RPS4Y1', 'AGR2', 'FN1', 'TGFB1', 'EPCAM', 'KRT7', 'S100A14', 'GPX2', 'KRT18',
'IFI27', 'TACSTD2']
```

```
[ ]: X_external = external_expression_clean.copy()
```

```
missing_cols = [
    col for col in feature_names
    if col not in X_external.columns
]
```

```
missing_df = pd.DataFrame(
    0,
    index=X_external.index,
    columns=missing_cols
)
```

```
X_external = pd.concat(
    [X_external, missing_df],
    axis=1
)
```

```
X_external = X_external[feature_names]
```

```
print(X_external.shape)
```

(1684, 10009)

```
[ ]: external_predictions = ridge_model.predict(
    X_external
)
```

```
print(external_predictions[:20])
```

```
[ 1.61382364  2.36950635  2.30647299  2.71946314  1.1233334  0.13623469
 0.98225065  2.27867628  0.85616978 -0.31266731  1.23486469  2.64899008
 2.02854299  2.18260966  2.56023872 -1.27756429  2.44219187  2.29725168
 1.81141286  2.39964726]
```

```
[ ]: plt.figure(figsize=(8,5))

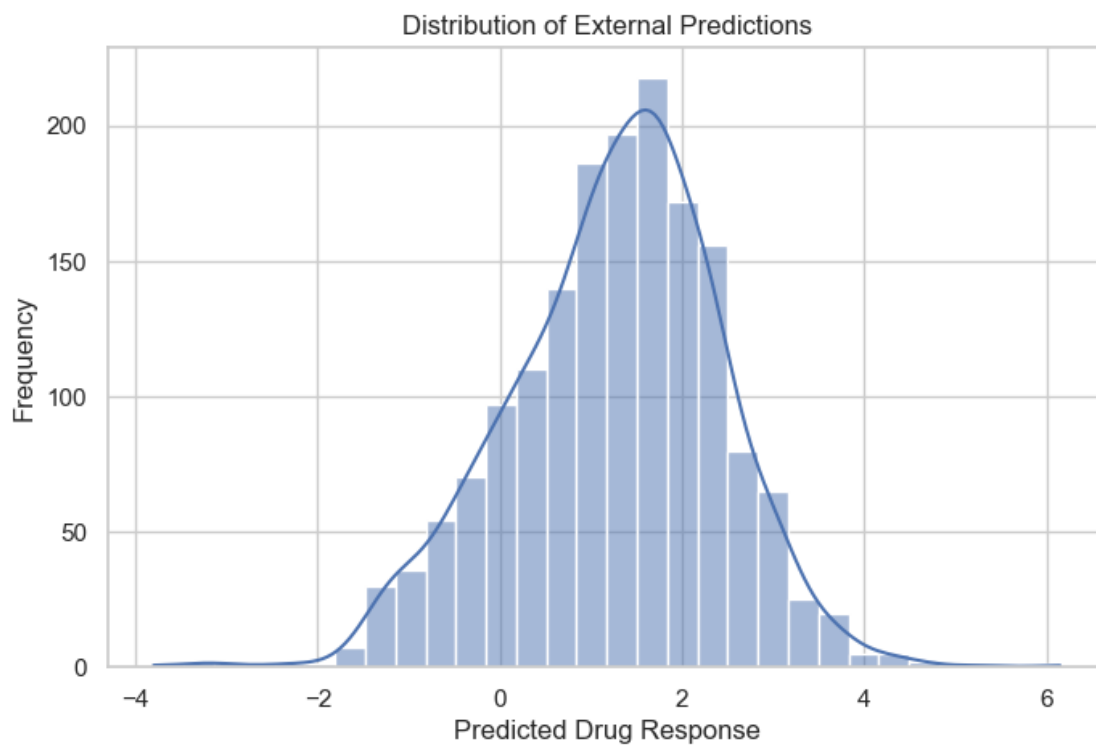
sns.histplot(
    external_predictions,
    bins=30,
    kde=True
)

plt.title(
    "Distribution of External Predictions"
)

plt.xlabel("Predicted Drug Response")

plt.ylabel("Frequency")

plt.show()
```



```
[ ]: plt.figure(figsize=(7,5))

plt.scatter(
    range(200),
    external_predictions[:200]
)
```

```

)

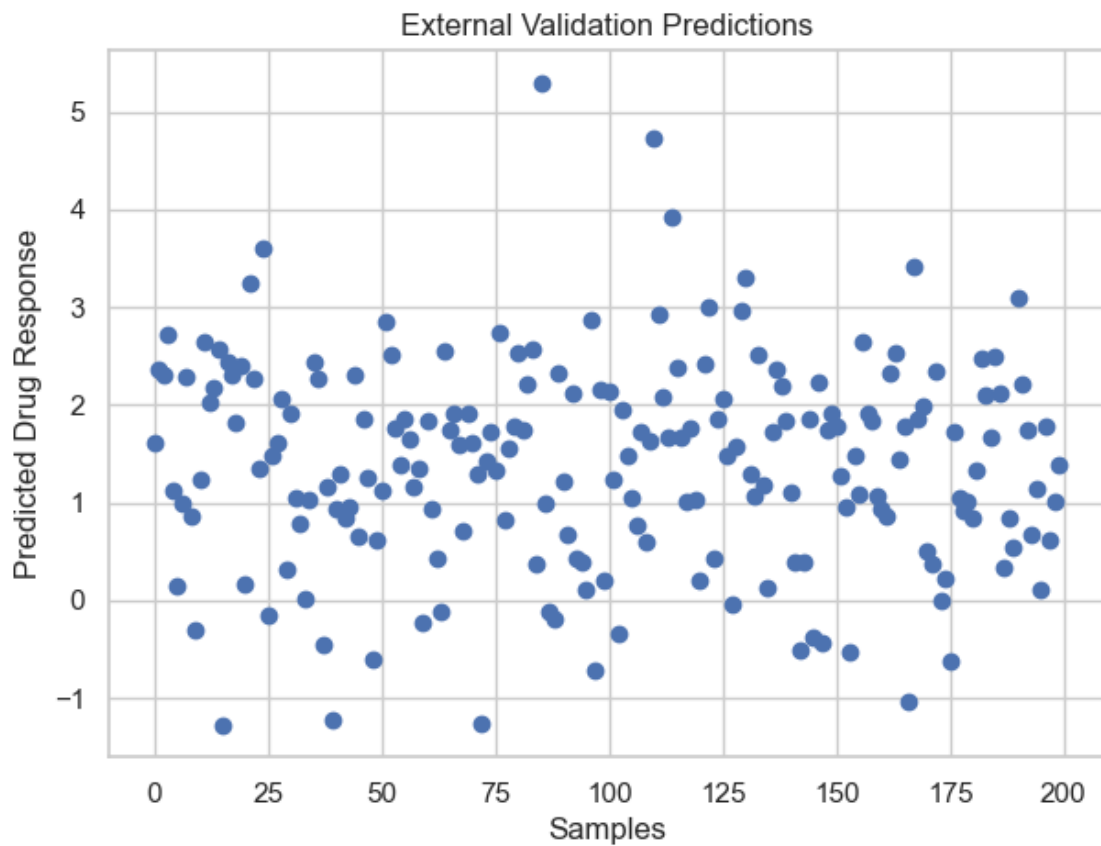
plt.title(
    "External Validation Predictions"
)

plt.xlabel("Samples")

plt.ylabel("Predicted Drug Response")

plt.show()

```



```

[ ]: residuals = (
    external_predictions
    - np.mean(external_predictions)
)

plt.figure(figsize=(7,5))

sns.scatterplot(

```



```

    x=external_predictions,
    y=residuals
)

plt.axhline(
    0,
    color='red',
    linestyle='--'
)

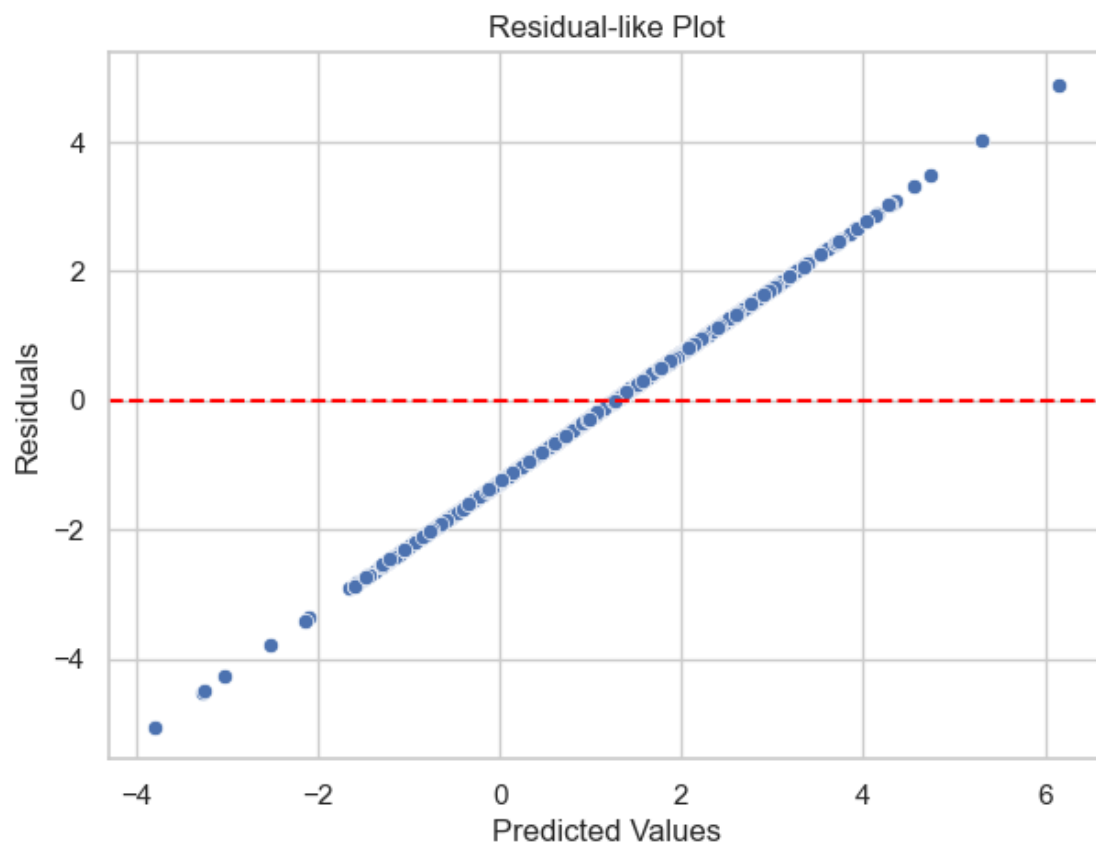
plt.title(
    "Residual-like Plot"
)

plt.xlabel("Predicted Values")

plt.ylabel("Residuals")

plt.show()

```



```
[ ]: from sklearn.model_selection import cross_val_score

sample_external = X_external.sample(
    300,
    random_state=42
)

sample_predictions = ridge_model.predict(
    sample_external
)

cv_scores = cross_val_score(
    ridge_model,
    sample_external,
    sample_predictions,
    cv=5,
    scoring="r2"
)

print("Cross-validation scores:")
print(cv_scores)

print("Mean CV score:")
print(cv_scores.mean())
```

Cross-validation scores:

[0.7427784 0.57151667 0.75465394 0.65409049 0.65183131]

Mean CV score:

0.6749741640473345

Deep Learning Using PyTorch Anar Aldhwaela

```
[1]: !pip install torch torchvision torchaudio
```

Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)

Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cpu)

Requirement already satisfied: torchaudio in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in
 /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
 Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
 (from torch) (3.1.6)
 Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-
 packages (from torch) (2025.3.0)
 Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
 (from torchvision) (2.0.2)
 Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
 /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in
 /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
 Requirement already satisfied: MarkupSafe>=2.0 in
 /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)

```
[2]: # Checking For PyTorch Installation Run
import torch
print(torch.__version__)
```

2.10.0+cpu

```
[3]: # Deep Learning(PyTorch)
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
```

```
[19]: # Data Loading and Exploring
ft = pd.read_csv("/content/The_Normalized_Genomic_Feature_Matrix.csv")
ft.head()
ft.tail()
```

```
[19]:
```

	VIM	KRT19	SPARC	RPL36A-HNRNPH2	S100P	C19orf33	\
10292	-1.696338	0.738997	-1.199510	-1.175804	1.995273	0.779535	
10293	-1.747956	1.038061	-1.196029	-1.293796	1.190833	0.801042	
10294	-1.758508	1.256902	-1.169459	-1.177753	1.559809	0.975188	
10295	-1.357700	1.564890	-1.133025	-1.038294	1.577686	1.351747	
10296	-1.747956	1.025295	-1.185826	-1.325515	1.663572	1.235911	

	FXVD3	UCHL1	LCN2	KRT8	...	DRUG_1034	DRUG_1049	\
10292	1.671577	-0.935427	-0.620436	0.876444	...	0	0	
10293	1.765242	-1.278236	0.540884	1.113539	...	0	0	
10294	1.235921	-1.169225	1.424821	1.357347	...	0	0	
10295	1.811797	-1.186292	1.904198	1.514362	...	0	0	
10296	1.150886	-0.690836	0.680671	1.064184	...	0	0	

	DRUG_1083	DRUG_1194	DRUG_1463	DRUG_1786	DRUG_1922	DRUG_1936	\
10292	0	0	0	0	0	0	
10293	0	0	0	0	0	0	

10294	0	0	0	0	0	0
10295	0	0	0	0	0	0
10296	0	0	0	0	0	0

	DRUG_2040	LN_IC50
10292	1	-0.732719
10293	1	1.027171
10294	1	3.247533
10295	1	1.633963
10296	1	0.670544

[5 rows x 10010 columns]

```
[18]: # Explore Data Features and Shape
print(f"   Shape: {ft.shape}")
print(f"   Columns: {ft.columns[-5:]}")
```

```
Shape: (820, 10010)
Columns: Index(['DRUG_1786', 'DRUG_1922', 'DRUG_1936', 'DRUG_2040',
'LN_IC50'], dtype='object')
```

```
[55]: # Summing All Missing Value In The Dataset
print(ft.isnull().sum().sum())
```

0

```
[46]: # Data Head After Dealing With Missing Values
print("data after dropping missing values:")
ftdrop.head()
```

data after dropping missing values:

```
[46]:      VIM      KRT19      SPARC  RPL36A-HNRNPH2      S100P  C19orf33      FXYD3  \
0  0.853036 -0.546034  0.681859      0.835942 -0.584490 -0.937651 -0.799620
1  0.052534 -1.139735  0.540839     -0.925476 -0.723836 -1.011830 -0.837324
2  1.056809 -1.068787  0.838472     -0.786594 -0.471451 -1.092821 -0.819758
3  0.223612 -1.136597  0.764662     -0.984604 -0.897553 -1.092821 -0.863326
4  0.057835 -0.682519  0.479214      0.961972 -0.499027 -0.855119 -0.624932
```

	UCHL1	LCN2	KRT8	...	DRUG_1034	DRUG_1049	DRUG_1083	\
0	1.099509	-0.725667	-0.284214	...	0	0	0	
1	0.949935	-0.791837	-0.709753	...	0	0	0	
2	1.338962	-0.725667	-1.492431	...	0	0	0	
3	1.286780	-0.804627	-1.705512	...	0	0	0	
4	1.278260	-0.414718	-1.113203	...	0	0	0	

	DRUG_1194	DRUG_1463	DRUG_1786	DRUG_1922	DRUG_1936	DRUG_2040	LN_IC50
0	0	0	0	0	0	0	1.180880
1	0	0	0	0	0	0	0.767377

2	0	0	0	0	0	0	0.943167
3	0	0	0	0	0	0	3.024536
4	0	0	0	0	0	0	0.640129

[5 rows x 10010 columns]

```
[21]: # Dropping Missing Values
ftdrop = ft.dropna()
print(f"Shape after dropping NaN: {ftdrop.shape}")
```

Shape after dropping NaN: (10297, 10010)

```
[56]: # Input features
X = ft.drop("LN_IC50", axis=1)

# Target variable
y = ft["LN_IC50"]
```

```
[24]: # Converting pandas DF to NumPy Arrays
# PyTorch Is more Compatible With Arrays
X = X.values
y = y.values
```

```
[25]: # Splitting Dataset Into Training And Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
[26]: # Converting Training and Testing Features Into PyTorch Tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)

# Converting The Target Values Into Tensors
# view(-1,1) Is To Reshape Data Into One Column
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1,1)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).view(-1,1)
```

```
[27]: # Combining Features And Targets Into One Datasets
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
```

```
[28]: # Creating Batches For Training
# And Randomizing Data Order In Each Epoch
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
```

```

        shuffle=True
    )

```

```

[29]: # Defining The Neural Network Model
class IC50Predictor(nn.Module):

    def __init__(self, input_size):
        # Initializing Class
        super(IC50Predictor, self).__init__()

        # Defining Network Layers
        self.network = nn.Sequential(

            # First Hidden Layer
            nn.Linear(input_size, 512),
            nn.ReLU(),
            # Dropping out can reduces the
            # Chances Of Overfitting
            nn.Dropout(0.3),

            # 2nd Hidden Layer
            nn.Linear(512, 256),
            nn.ReLU(),

            nn.Dropout(0.3),
            # 3rd Hidden Layer
            nn.Linear(256, 128),
            nn.ReLU(),

            # Output Layer
            nn.Linear(128, 1)
        )

        # Forward Pass Through The Network
    def forward(self, x):
        return self.network(x)

```

```

[30]: # Number Of Input Features
input_size = X_train.shape[1]

```

```

[31]: # Creating Model Object
model = IC50Predictor(input_size)
# Checking For The Model Architecture
print(model)

```

```

IC50Predictor(
  (network): Sequential(
    (0): Linear(in_features=10009, out_features=512, bias=True)
    (1): ReLU()
  )
)

```

```

        (2): Dropout(p=0.3, inplace=False)
        (3): Linear(in_features=512, out_features=256, bias=True)
        (4): ReLU()
        (5): Dropout(p=0.3, inplace=False)
        (6): Linear(in_features=256, out_features=128, bias=True)
        (7): ReLU()
        (8): Linear(in_features=128, out_features=1, bias=True)
    )
)

```

```

[32]: # Define Loss Function
      # MSE Is Usually Used For Regression Problems
      criterion = nn.MSELoss()

```

```

[33]: # Define Optimizer
      # Adam Updates Model Weights During Training
      optimizer = torch.optim.Adam(
          model.parameters(),
          lr=0.001
      )

```

```

[34]: # Number of training cycles
      epochs = 100

      # Store training loss
      losses = []

      for epoch in range(epochs):

          model.train()

          running_loss = 0

          for batch_X, batch_y in train_loader:

              # Reset gradients
              optimizer.zero_grad()

              # Forward pass
              outputs = model(batch_X)

              # Calculate loss
              loss = criterion(outputs, batch_y)

              # Backpropagation
              loss.backward()

```

```

        # Update weights
        optimizer.step()

        running_loss += loss.item()

    epoch_loss = running_loss / len(train_loader)

    losses.append(epoch_loss)

    print(f"Epoch {epoch+1}/{epochs}, Loss: {epoch_loss:.4f}")

```

```

Epoch 1/100, Loss: 6.1367
Epoch 2/100, Loss: 5.6486
Epoch 3/100, Loss: 5.5033
Epoch 4/100, Loss: 5.3193
Epoch 5/100, Loss: 5.2783
Epoch 6/100, Loss: 5.1635
Epoch 7/100, Loss: 5.1304
Epoch 8/100, Loss: 5.0905
Epoch 9/100, Loss: 5.0147
Epoch 10/100, Loss: 4.9824
Epoch 11/100, Loss: 4.9509
Epoch 12/100, Loss: 4.9048
Epoch 13/100, Loss: 4.8720
Epoch 14/100, Loss: 4.7913
Epoch 15/100, Loss: 4.7203
Epoch 16/100, Loss: 4.7284
Epoch 17/100, Loss: 4.6241
Epoch 18/100, Loss: 4.6362
Epoch 19/100, Loss: 4.4801
Epoch 20/100, Loss: 4.4395
Epoch 21/100, Loss: 4.4082
Epoch 22/100, Loss: 4.3355
Epoch 23/100, Loss: 4.2080
Epoch 24/100, Loss: 4.0600
Epoch 25/100, Loss: 3.9700
Epoch 26/100, Loss: 3.8609
Epoch 27/100, Loss: 3.7786
Epoch 28/100, Loss: 3.6520
Epoch 29/100, Loss: 3.5943
Epoch 30/100, Loss: 3.4574
Epoch 31/100, Loss: 3.3530
Epoch 32/100, Loss: 3.2921
Epoch 33/100, Loss: 3.1452
Epoch 34/100, Loss: 3.1108
Epoch 35/100, Loss: 2.9488
Epoch 36/100, Loss: 2.8780
Epoch 37/100, Loss: 2.9238

```


Epoch 38/100, Loss: 2.7483
Epoch 39/100, Loss: 2.6989
Epoch 40/100, Loss: 2.6432
Epoch 41/100, Loss: 2.5618
Epoch 42/100, Loss: 2.4423
Epoch 43/100, Loss: 2.4338
Epoch 44/100, Loss: 2.3657
Epoch 45/100, Loss: 2.3342
Epoch 46/100, Loss: 2.2658
Epoch 47/100, Loss: 2.1556
Epoch 48/100, Loss: 2.1357
Epoch 49/100, Loss: 2.1950
Epoch 50/100, Loss: 2.0568
Epoch 51/100, Loss: 2.0509
Epoch 52/100, Loss: 2.0782
Epoch 53/100, Loss: 2.0339
Epoch 54/100, Loss: 2.0398
Epoch 55/100, Loss: 1.9380
Epoch 56/100, Loss: 1.9373
Epoch 57/100, Loss: 1.8466
Epoch 58/100, Loss: 1.9089
Epoch 59/100, Loss: 1.8346
Epoch 60/100, Loss: 1.8989
Epoch 61/100, Loss: 1.8144
Epoch 62/100, Loss: 1.8132
Epoch 63/100, Loss: 1.7657
Epoch 64/100, Loss: 1.6827
Epoch 65/100, Loss: 1.8160
Epoch 66/100, Loss: 1.8031
Epoch 67/100, Loss: 1.8110
Epoch 68/100, Loss: 1.7588
Epoch 69/100, Loss: 1.7336
Epoch 70/100, Loss: 1.7305
Epoch 71/100, Loss: 1.6630
Epoch 72/100, Loss: 1.6874
Epoch 73/100, Loss: 1.7108
Epoch 74/100, Loss: 1.6700
Epoch 75/100, Loss: 1.7149
Epoch 76/100, Loss: 1.6603
Epoch 77/100, Loss: 1.6659
Epoch 78/100, Loss: 1.6173
Epoch 79/100, Loss: 1.5839
Epoch 80/100, Loss: 1.6312
Epoch 81/100, Loss: 1.5933
Epoch 82/100, Loss: 1.5802
Epoch 83/100, Loss: 1.6031
Epoch 84/100, Loss: 1.6305
Epoch 85/100, Loss: 1.6151

```
Epoch 86/100, Loss: 1.5794
Epoch 87/100, Loss: 1.6133
Epoch 88/100, Loss: 1.5401
Epoch 89/100, Loss: 1.5212
Epoch 90/100, Loss: 1.4509
Epoch 91/100, Loss: 1.4600
Epoch 92/100, Loss: 1.4676
Epoch 93/100, Loss: 1.4768
Epoch 94/100, Loss: 1.5602
Epoch 95/100, Loss: 1.5174
Epoch 96/100, Loss: 1.4859
Epoch 97/100, Loss: 1.5589
Epoch 98/100, Loss: 1.5575
Epoch 99/100, Loss: 1.5173
Epoch 100/100, Loss: 1.4432
```

```
[35]: # Setting The Model To Evaluation Mode
model.eval()

# Disable Gradient Calculations During Prediction
with torch.no_grad():
    # Predict LN_IC50 Values
    predictions = model(X_test_tensor)

# Convert predictions to NumPy
predictions = predictions.numpy()
```

```
[36]: # Calculate Root Mean Squared Error
rmse = np.sqrt(
    mean_squared_error(y_test, predictions)
)

print("RMSE:", rmse)
```

RMSE: 1.2684962585420105

```
[37]: # Calculate R^2 Score
# Measures Prediction Accuracy
r2 = r2_score(y_test, predictions)

print("R^2 Score:", r2)
```

R^2 Score: 0.7257607999797819

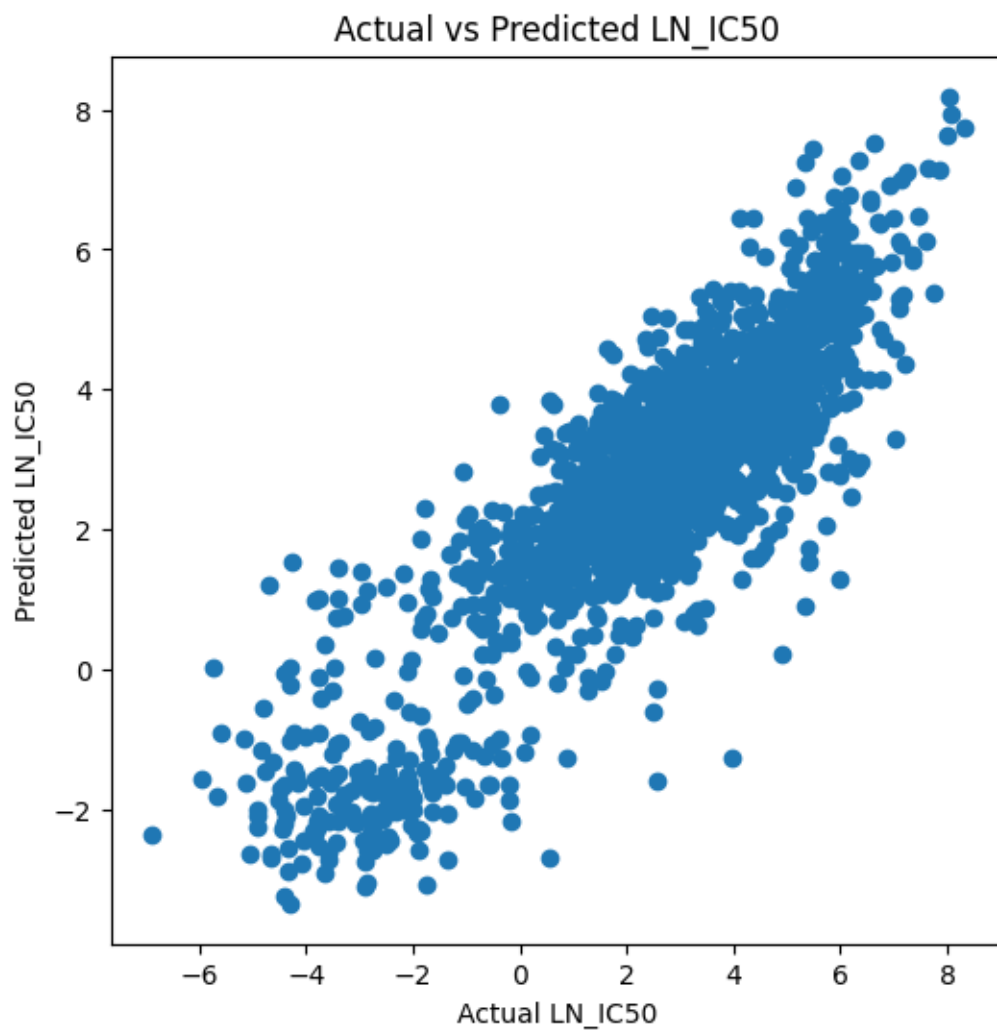
```
[38]: # Create Scatter Plot
plt.figure(figsize=(6,6))

# Plotting Actual vs Predicted Values
plt.scatter(y_test, predictions)
```

```
plt.xlabel("Actual LN_IC50") # X-axis Label
plt.ylabel("Predicted LN_IC50") # Y-axis Label

# Plot Title
plt.title("Actual vs Predicted LN_IC50")

#Display Plot
plt.show()
```



```
[47]: print(model)
```

```
IC50Predictor(
  (network): Sequential(
    (0): Linear(in_features=10009, out_features=512, bias=True)
```

```

(1): ReLU()
(2): Dropout(p=0.3, inplace=False)
(3): Linear(in_features=512, out_features=256, bias=True)
(4): ReLU()
(5): Dropout(p=0.3, inplace=False)
(6): Linear(in_features=256, out_features=128, bias=True)
(7): ReLU()
(8): Linear(in_features=128, out_features=1, bias=True)
)
)

```

Feature Importance of the Machine and Deep learning Models Machine Learning:

```

[16]: Linear_Model = joblib.load("linear_regression_model.pkl")
Best_Ridge_Model = joblib.load("ridge_regression_model.pkl")
Best_RF_Model = joblib.load("random_forest_model.pkl")
Best_XGB_Model = joblib.load("xgboost_model.pkl")
# Loading trained models to work with right away

```

```

C:\Users\JA\anaconda3\Lib\site-packages\sklearn\base.py:376:
InconsistentVersionWarning: Trying to unpickle estimator LinearRegression from
version 1.5.1 when using version 1.4.2. This might lead to breaking code or
invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model\_persistence.html#security-maintainability-
limitations
  warnings.warn(
C:\Users\JA\anaconda3\Lib\site-packages\sklearn\base.py:376:
InconsistentVersionWarning: Trying to unpickle estimator Ridge from version
1.5.1 when using version 1.4.2. This might lead to breaking code or invalid
results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model\_persistence.html#security-maintainability-
limitations
  warnings.warn(
C:\Users\JA\anaconda3\Lib\site-packages\sklearn\base.py:376:
InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeRegressor
from version 1.5.1 when using version 1.4.2. This might lead to breaking code or
invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model\_persistence.html#security-maintainability-
limitations
  warnings.warn(
C:\Users\JA\anaconda3\Lib\site-packages\sklearn\base.py:376:
InconsistentVersionWarning: Trying to unpickle estimator RandomForestRegressor
from version 1.5.1 when using version 1.4.2. This might lead to breaking code or
invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model\_persistence.html#security-maintainability-
limitations
  warnings.warn(

```

```
[18]: # Function to display and visualize top important features
def plot_top_features(importance_df, model_name, top_n=10):

    top_features = importance_df.sort_values(
        by="Importance",
        ascending=False
    ).head(top_n)

    print(f"\nTop {top_n} Important Features - {model_name}")
    display(top_features)

    plt.figure(figsize=(8, 5))

    sns.barplot(
        data=top_features,
        x="Importance",
        y="Feature"
    )

    plt.title(f"Top {top_n} Important Features - {model_name}")
    plt.xlabel("Importance")
    plt.ylabel("Feature")

    plt.show()

    return top_features
```

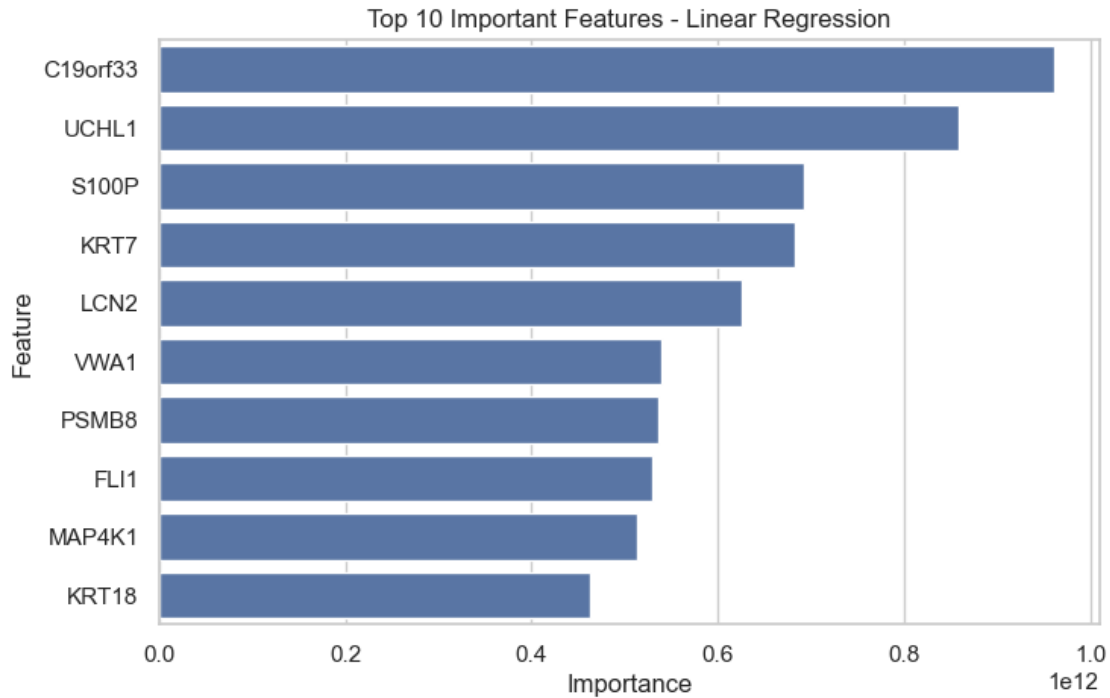
```
[32]: # Feature importance for Linear Regression using coefficient values
linear_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": abs(Linear_Model.coef_)
})

top_linear_features = plot_top_features(
    linear_importance,
    "Linear Regression"
)
```

Top 10 Important Features - Linear Regression

	Feature	Importance
5	C19orf33	9.616524e+11
7	UCHL1	8.587364e+11
4	S100P	6.918402e+11
15	KRT7	6.835688e+11
8	LCN2	6.257727e+11
954	VWA1	5.396848e+11

924	PSMB8	5.359654e+11
953	FLI1	5.292355e+11
889	MAP4K1	5.136104e+11
18	KRT18	4.626517e+11



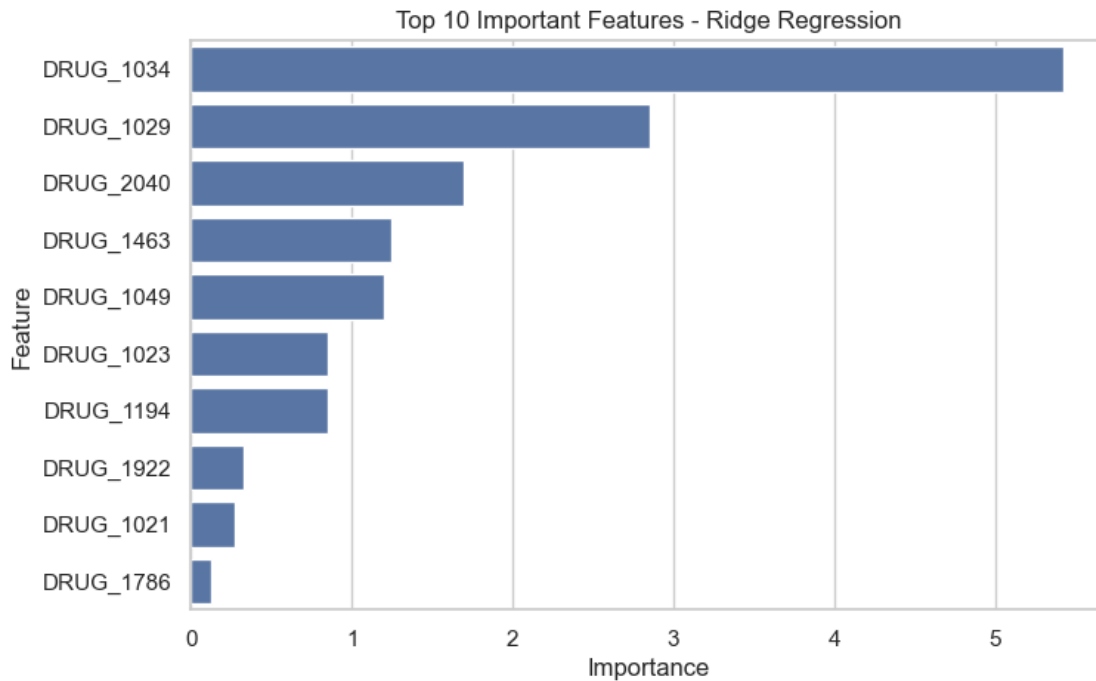
```
[34]: # Feature importance for Ridge Regression using coefficient values
ridge_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": abs(Best_Ridge_Model.coef_)
})

top_ridge_features = plot_top_features(
    ridge_importance,
    "Ridge Regression"
)
```

Top 10 Important Features - Ridge Regression

	Feature	Importance
10000	DRUG_1034	5.424043
9999	DRUG_1029	2.854597
10008	DRUG_2040	1.697024
10004	DRUG_1463	1.244876
10001	DRUG_1049	1.196776

9998	DRUG_1023	0.854708
10003	DRUG_1194	0.850726
10006	DRUG_1922	0.329079
9997	DRUG_1021	0.269818
10005	DRUG_1786	0.127855



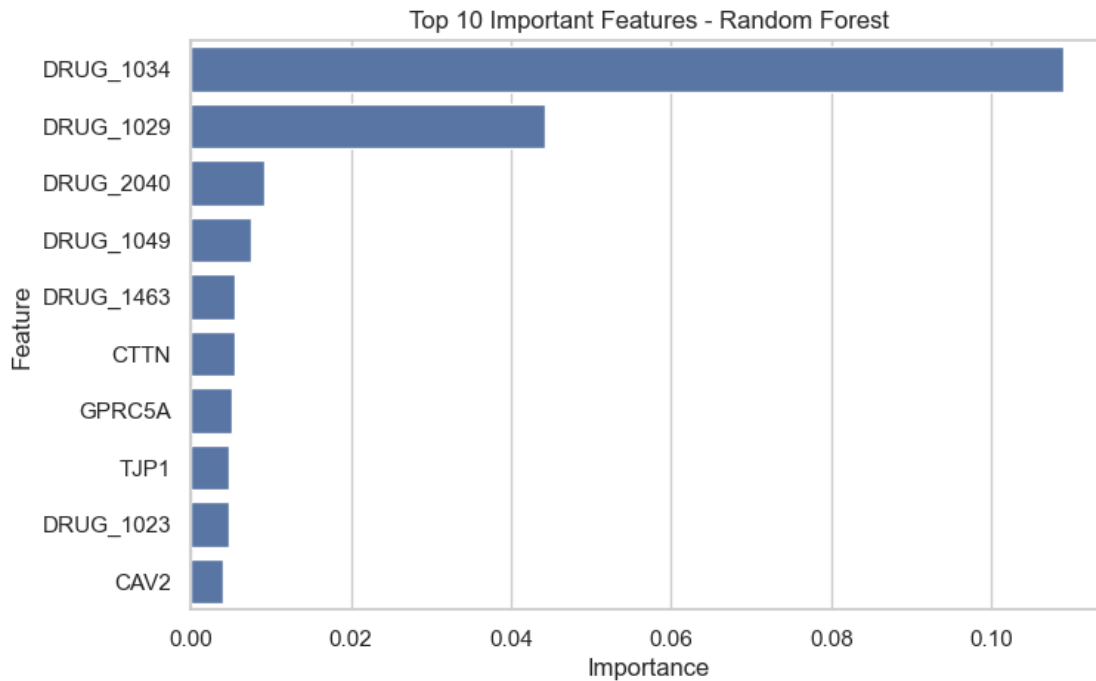
```
[36]: # Feature importance for Random Forest using built-in feature importance scores
rf_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": Best_RF_Model.feature_importances_
})

top_rf_features = plot_top_features(
    rf_importance,
    "Random Forest"
)
```

Top 10 Important Features - Random Forest

	Feature	Importance
10000	DRUG_1034	0.108976
9999	DRUG_1029	0.044160
10008	DRUG_2040	0.009223
10001	DRUG_1049	0.007417
10004	DRUG_1463	0.005419

1310	CTTN	0.005402
122	GPRC5A	0.005107
1502	TJP1	0.004642
9998	DRUG_1023	0.004641
153	CAV2	0.003997



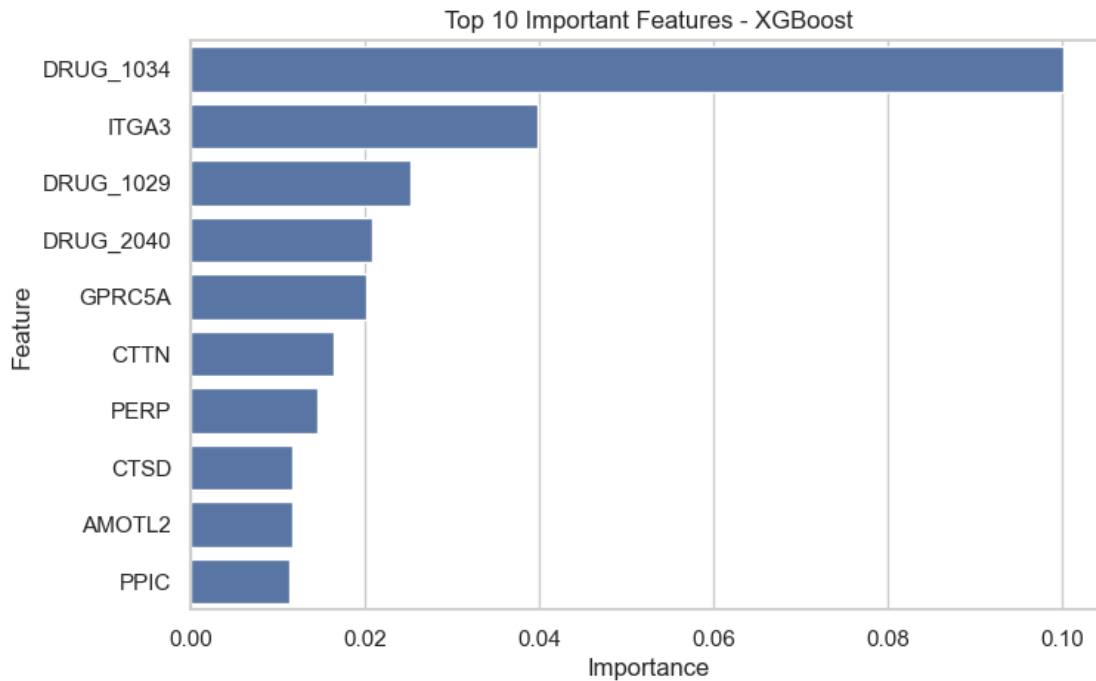
```
[38]: # Feature importance for XGBoost using built-in feature importance scores
xgb_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": Best_XGB_Model.feature_importances_
})

top_xgb_features = plot_top_features(
    xgb_importance,
    "XGBoost"
)
```

Top 10 Important Features - XGBoost

	Feature	Importance
10000	DRUG_1034	0.100110
187	ITGA3	0.039835
9999	DRUG_1029	0.025187
10008	DRUG_2040	0.020800
122	GPRC5A	0.020145

1310	CTTN	0.016407
340	PERP	0.014515
2479	CTSD	0.011678
558	AMOTL2	0.011551
819	PPIC	0.011255



Deep Learning:

```
[57]: weights = model.network[0].weight.detach().cpu().numpy()

# Calculate The Importance (mean absolute weight)
importance_scores = np.mean(np.abs(weights), axis=0)

# Create The Importance Table
feature_names = ft.drop(columns=['LN_IC50']).columns

importance_ft = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importance_scores
})

# Sort and show the top 10
importance_ft = importance_ft.sort_values(by='Importance', ascending=False)
print(importance_ft.head(10))
```

Feature	Importance
---------	------------

9999	DRUG_1029	10.819161
10000	DRUG_1034	9.894964
10001	DRUG_1049	5.180589
10004	DRUG_1463	5.122636
9998	DRUG_1023	3.790335
10008	DRUG_2040	3.618149
10003	DRUG_1194	3.101679
9997	DRUG_1021	2.181860
10005	DRUG_1786	1.825873
10007	DRUG_1936	1.736247

Biological Interpretation and Feature Importance of the Machine and Deep Learning Models:

```
[52]: # Show the DRUG_IDs and DRUG_NAMES side by side
Drug_Mapping = Processed_Data[['DRUG_ID', 'DRUG_NAME']].drop_duplicates()
display(Drug_Mapping.sort_values(by='DRUG_ID'))
```

	DRUG_ID	DRUG_NAME
0	1021	Axitinib
936	1023	GW441756
1864	1029	Motesanib
2789	1034	Staurosporine
3732	1049	PD173074
4674	1083	Crizotinib
5615	1194	SB505124
5872	1463	AZD1332
6570	1786	AZD4547
7504	1922	Cediranib
8437	1936	Savolitinib
9369	2040	Foretinib

Machine Learning:

The machine learning models showed that both drug identity and gene expression profiles contributed to the prediction of LN_IC50. Feature importance analysis identified several RTK inhibitor drugs, including Motesanib (DRUG_1029), Staurosporine (DRUG_1034), PD173074 (DRUG_1049), Crizotinib (DRUG_1083), and Cediranib (DRUG_1922), as highly influential predictors. This suggests that differences between RTK-targeting drugs strongly affected the predicted drug sensitivity values across cancer cell lines. Several biologically relevant genes also appeared among the important genomic predictors, including VIM, KRT8, KRT19, S100P, UCHL1, and LCN2. These genes are associated with epithelial-mesenchymal transition (EMT), tumor progression, invasion, metastasis, and mechanisms of treatment resistance in cancer biology. Their importance suggests that transcriptomic differences between cancer cell lines influenced sensitivity to RTK-targeted therapies. Overall, the machine learning models combined transcriptomic and pharmacogenomic information to predict LN_IC50 values and identify biologically meaningful patterns associated with RTK drug response. However, these results represent predictive associations rather than clinically validated biomarkers and would require further biological validation.

Deep Learning:

The deep learning model achieved strong predictive performance for LN_IC50 using both gene expression features and encoded RTK drug variables. Feature importance analysis showed that several RTK-targeting drugs, including Motesanib (DRUG_1029), Staurosporine (DRUG_1034), PD173074 (DRUG_1049), and Crizotinib (DRUG_1083), contributed strongly to prediction, indicating that drug-specific effects influenced cancer cell sensitivity patterns across the dataset. In addition, biologically important genes such as VIM, KRT8, KRT19, S100P, and LCN2 were identified as relevant transcriptomic predictors. These genes are associated with epithelial–mesenchymal transition (EMT), cellular differentiation, tumor progression, invasion, and drug resistance mechanisms. The deep learning model likely captured complex nonlinear relationships between these molecular features and RTK-targeted drug response. Overall, integrating transcriptomic profiles with pharmacogenomic drug information improved predictive performance and provided biologically meaningful insight into cancer drug sensitivity. Although the identified features may represent important predictive markers, additional experimental and clinical validation would be necessary before considering them confirmed therapeutic biomarkers.